CSE 312
Operating Systems

Spring 2026

File Systems

File Systems

- One of the most visible pieces of the OS
- Contributes significantly to usability (or the lack thereof)

Design Choices

Files and File Systems

- What's a file?
  - You all know what a file is. . .
- What's a file system?
  - A file system is a mapping of file names to a subset of a physical medium
- A file system is the OS abstraction for storage resources
  - File is a logical storage unit in the OS abstract interface for storage resources
    - Extension of address space (temporary files)
    - Non-volatile storage that survives the execution of an individual program (persistent files)
  - Directory is a logical "container" for a group of files

File Systems

Essential requirements for long-term information storage:

- It must be possible to store a very large amount of information
- The information must survive the termination of the process using it
- Multiple processes must be able to access the information concurrently

Example

How do you store a digital movie for 100 years?

IEEE Spectrum March 2014

- A single feature-length digital motion picture generates
  - 2 petabytes
  - ~ 500K DVDs
- Each year 700 new feature films are released…
- A single petabyte takes 90 days to transfer over 1 gigabit ethernet

File Systems

Think of a disk as a linear sequence of fixed-size blocks and supporting reading and writing of blocks.

Questions that quickly arise:

- How do you find information?
- How do you keep one user from reading another's data?
- How do you know which blocks are free?

Many Types of File Systems

- Unix file systems
- Windows file systems
- CDs
- DVDs
- DECtapes — ancient variant of magnetic tape that permitted seeking and overwriting individual blocks in the middle
- They all had variants

Design Decisions

- File names
- Hierarchical or non-hierarchical
- Types of access control
- Special media characteristics
- Available space
- Media speed
- Read-only or read-write
- Versioning
- Fault recovery
- Record-, byte-, or block-structured
- Many more

File Names

- Case sensitivity
- Extensions — permitted, required, uninterpreted, length
- Character set
- Uniform or non-uniform file-naming scheme

File Naming

Extension	Meaning
file.bak	Backup file
file.c	C source program
file.gif	Compuserve Graphical Interchange Format image
file.hlp	Help file
file.html	World Wide Web HyperText Markup Language document
file.jpg	Still picture encoded with the JPEG standard
file.mp3	Music encoded in MPEG layer 3 audio format
file.mpg	Movie encoded with the MPEG standard
file.o	Object file (compiler output, not yet linked)
file.pdf	Portable Document Format file
file.ps	PostScript file
file.tex	Input for the TEX formatting program
file.txt	General text file
file.zip	Compressed archive

Types of Names

- /this/is/a/unix.file.name
- q:\Windows likes\mOnOcaSE
- >Multics>uses<for>up-a-level
- os.360.looks.hierarchical.but.isnot(really)
- vms-host::device[direct.ory.list]name.type;1

Other Disks

- How are other disk drives named?
- On Unix, they're a subtree of the file system, via the mount command
- Multics had a single tree, but directories always lived on permanently-mounted disks
- On Windows and VMS, a drive letter is explicitly given
- On OS/360 and /370, you could use explicit disk identifiers or you could use the "system catalog" — a hierarchical structure for file names where the "directories" didn't need to live on the same disks as the files

Extensions

- On Unix and Multics, the kernel knows nothing of filename extensions; they're just characters. But some applications (make, the C compiler, etc., care)
- The old DOS file system used "8.3" format: 8-character filenames, with optional 3-character extension
- OS/360 confused extensions with directory levels

Media Characteristics

- Today's disks have fixed-size, 512-byte sectors
- Old mainframe disks had variable-size sectors, selected by the application
- CDs, even if writable, aren't block-writable the way disks are
- Blocks on flash disks can only be written a certain number of times before they wear out
- WORM disk blocks can be written and deleted, but not overwritten

Space and Speed

- How big can the file system be? How small must it be?
- Is space really tight? Can you trade space for speed?
- How large can files be? What is a typical size?

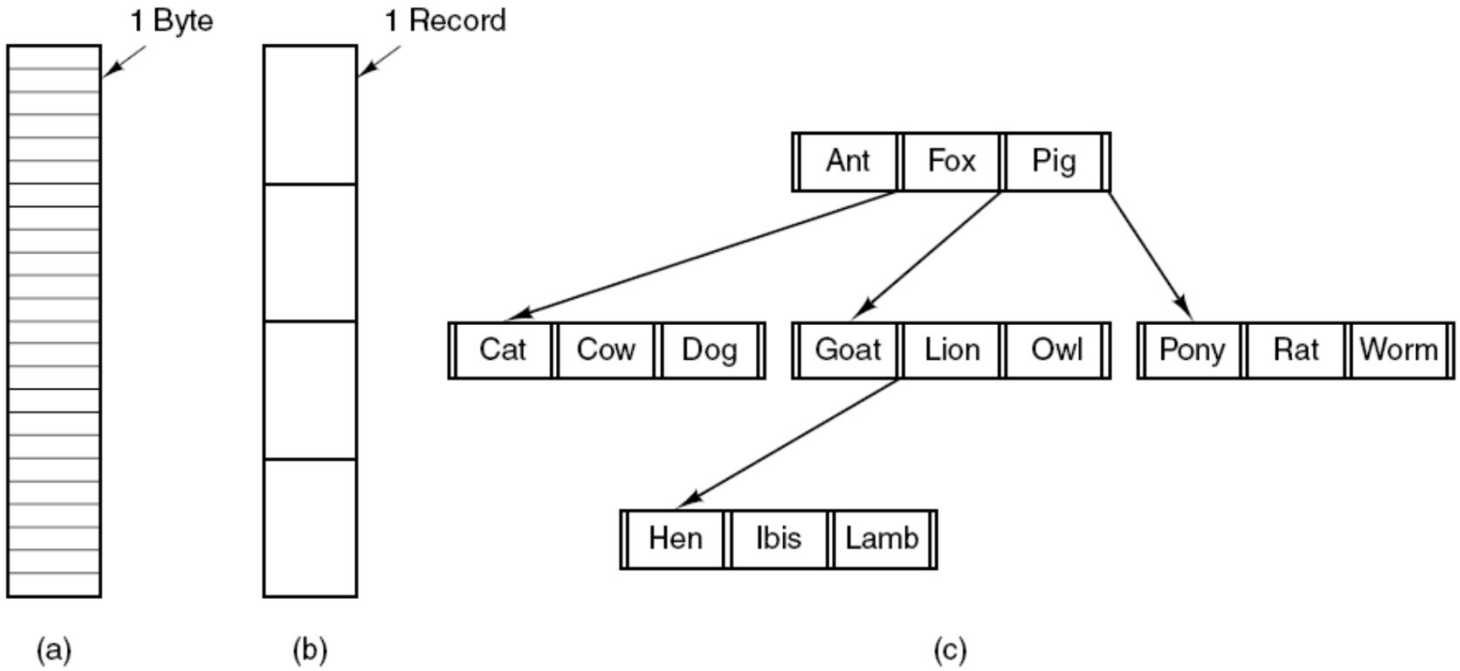
Versioning

- Some file systems (VMS, Tenex, TOPS-20) include a version number as part of the file name
- Plan 9 provides file system views at any arbitrary point in time

File Structures

- Old systems: files were sequences of records, initially images of punch cards or printer lines
- Unix: randomly-addressable sequences of bytes
- Page-oriented file systems: sequence of page-sized blocks

File Structure



Three kinds of files. (a) Byte sequence. (b) Record sequence. (c) Tree.

How Many Names?

- Can files have more than one name?
- What about directories?
- Are all names equal?
- How do you keep the file system graph acyclic? Or is it just a tree?

Underlying Hardware

Underlying Disk Hardware

- Today's disks use fixed-size blocks; some older disks did not
- IBM disks supported hardware search keys — you could seek to a track, then look for that key somewhere on the track
- Goal was good hardware support for databases — offload the CPU, by having the disks do more work

Supporting Such Disks

- Space allocation has to support track granularity
- Underlying I/O primitives have to support that mode of access
- If we mix block-structured files and search key files on the same disk, do we have to support tracks with both kinds of blocks?

Speed and Optimization

- Does the speed of the disk affect the file system design?
- What do we really know about the hardware?
- Old disks were based on cylinders, heads, and sectors; newer disks lie to the OS
- How do keep a file localized? Do we need to?

Interface to Lower Layers

- What is the interface to lower layers?
- Unix assumes “read/write block N”
- Multics has none; it’s just the paging system
- OS/360 permitted very general I/O requests

Space Allocation

Space Allocation

- What is our unit of space allocation?
- Does the programmer have to reserve space in advance?
- How much does the OS have to know about the application's data formats?

Units of Space Allocation

- Some systems required space to be allocated in hardware-based units: tracks, cylinders, etc.
- With variable sector sizes, how you packed the data determined how much you could fit onto a track and hence onto a disk
- If you packed more records into a single block, there were fewer blocks and hence fewer interblock gaps on each track

Keeping Track

- The OS has to keep track of all allocated space: which files own which parts of the disk
- What data structure is best?
- Also need to keep track of free space on the disk
- Space allocation is similar to RAM allocation
- Space allocated to a file may or may not need to be contiguous, depending on other design decisions

Metadata

Metadata

- We need to store a lot of data about the file
- Most obvious: disk blocks
- Other data: file creator, date created/ modified/ accessed
- File permissions

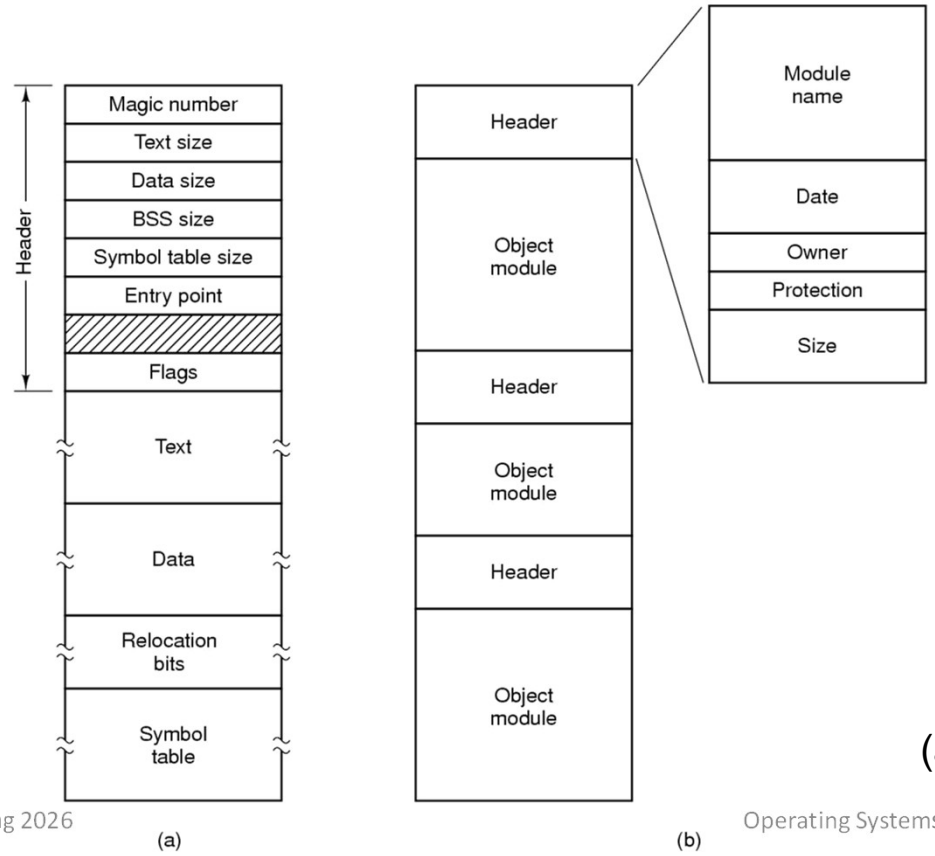
Other Metadata

- Read-only flag, independent of other file permissions
- “Hidden”
- System file
- Archive needed (similar to “page modified” flag)
- ASCII/Binary
- Record size

File Types

- Some operating systems consider some files to be special in some way
- On Unix, most I/O devices have filenames
- Directories are sometimes just files
- Some communications channels have filenames

File Types



(a) An executable file. (b) An archive.

Linux Metadata

```
struct stat {
    dev_t st_dev;               /* device */
    ino_t st_ino;               /* inode */
    mode_t st_mode;             /* protection */
    nlink_t st_nlink;    /* number of hard links */
    uid_t st_uid;               /* user ID of owner */
    gid_t st_gid;               /* group ID of owner */
    dev_t st_rdev;              /* device type (if inode device) */
    off_t st_size;              /* total size, in bytes */
    blksize_t st_blksize;       /* blocksize for filesystem I/O */
    blkcnt_t st_blocks;  /* number of blocks allocated */
    time_t st_atime;            /* time of last access */
    time_t st_mtime;            /* time of last modification */
    time_t st_ctime;            /* time of last status change */
};
```

File Attributes

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file was last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

Where is Metadata Stored?

- In the file?
- In the directory entry?
- Elsewhere?
- Split?

Metadata in the File

- (Sort of) done by Apple: resource and data forks
- Not very portable — when you copy the file to/from other systems, what happens to the metadata?
- No standardized metadata exchange format

Metadata in the Directory

- Speeds access to metadata
- Makes hard links difficult — need to keep copies of the metadata synchronized
- Makes directories larger; often, one doesn't need the metadata
- Many newer systems keep at least a few bits of metadata in the directory, notably file type —knowing if something is or isn't a directory speeds up tree walks considerably

Crash Recovery

Crash Recovery

- Must ensure that file systems are in a consistent state after a system crash
- Example: don’t write out directory entry before the metadata
- Example: File systems are generally trees, not graphs; make sure things always point somewhere sane
- What if the file has blocks but the freelist hasn’t been updated?
- Principle: order writes to ensure that the disk is always in a safe state

Repairing Damage

- At boot-time, run a consistency checker except after a clean shutdown
- Example: fsck (Unix) and scandisk (Windows)
- Force things to a safe state; move any referenced blocks to a recovery area

Log-Structured File Systems

- Instead of overwriting data, append the changes to a journaling area
- The file system is thus always consistent, as long as writes are properly ordered.
- Is that a reasonable assumption?

Modern Disks

- Modern disks do a lot of buffering
- Cache size on new Seagate drives: 2-16M bytes
- The drive will reorder writes to optimize seek times
- If a bad block has been relocated, you can't even predict when seeks will occur; only the drive knows
- What if the power fails when data is buffered?

File Operations

File Operations

The most common system calls relating to files:

- Create
- Delete
- Open
- Close
- Read
- Write
- Append
- Seek
- Get Attributes
- Set Attributes
- Rename

Example Program Using File System Calls

```
/* File copy program. Error checking and reporting is minimal. */

#include <sys/types.h>                  /* include necessary header files */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]);       /* ANSI prototype */

#define BUF_SIZE 4096                   /* use a buffer size of 4096 bytes */
#define OUTPUT_MODE 0700                /* protection bits for output file */

int main(int argc, char *argv[])
{
    int in_fd, out_fd, rd_count, wt_count;
    char buffer[BUF_SIZE];

    if (argc != 3) exit(1);             /* syntax error if argc is not 3 */

    /* Open the input file and create the output file */
    in_fd = open(argv[1], O_RDONLY);    /* open the source file */
    if (in_fd < 0) exit(2);             /* if it cannot be opened, exit */
    out_fd = creat(argv[2], OUTPUT_MODE); /* create the destination file */
    if (out_fd < 0) exit(3);            /* if it cannot be created, exit */
```

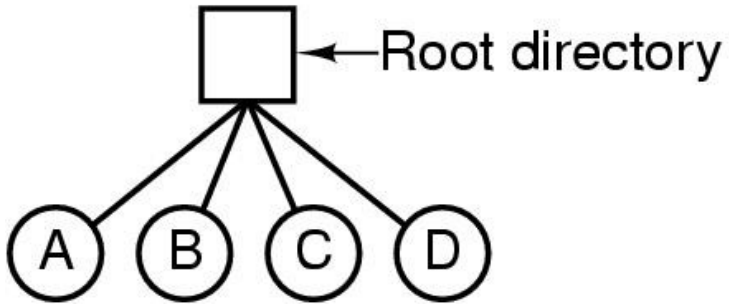
Example Program Using File System Calls

```
        /* Copy loop */
        while (TRUE) {
                rd_count = read(in_fd, buffer, BUF_SIZE); /* read a block of data */
        if (rd_count <= 0) break;                       /* if end of file or error, exit loop */
                wt_count = write(out_fd, buffer, rd_count); /* write data */
                if (wt_count <= 0) exit(4);             /* wt_count <= 0 is an error */
        }

        /* Close the files */
        close(in_fd);
        close(out_fd);
        if (rd_count == 0)                              /* no error on last read */
                exit(0);
        else
                exit(5);                                /* error on last read */
}
```

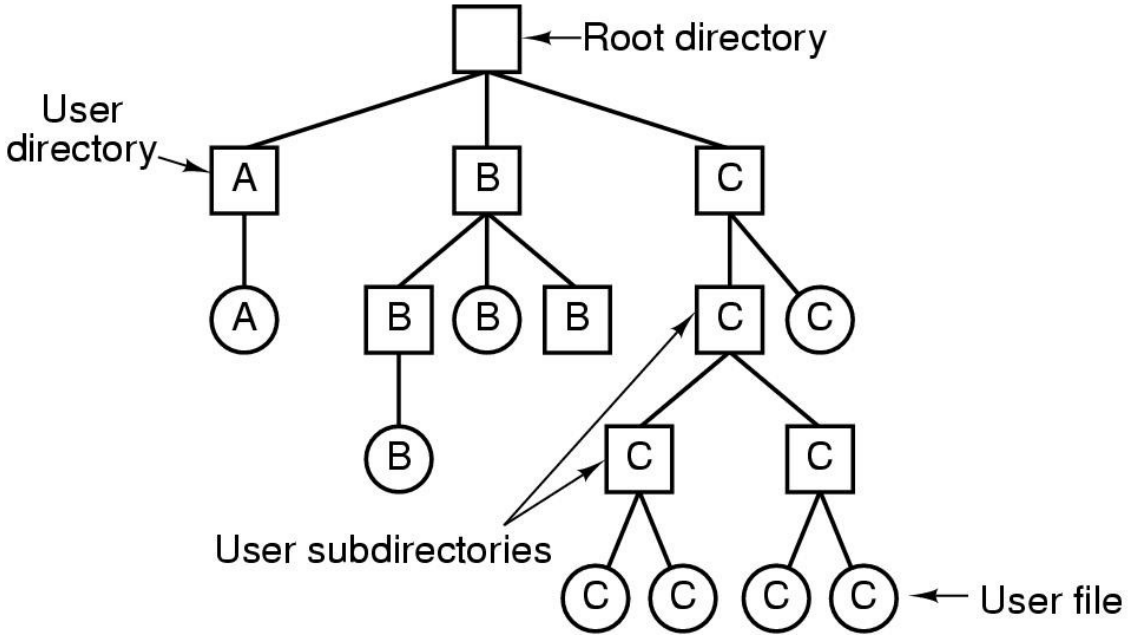
Directories

Hierarchical Directory Systems



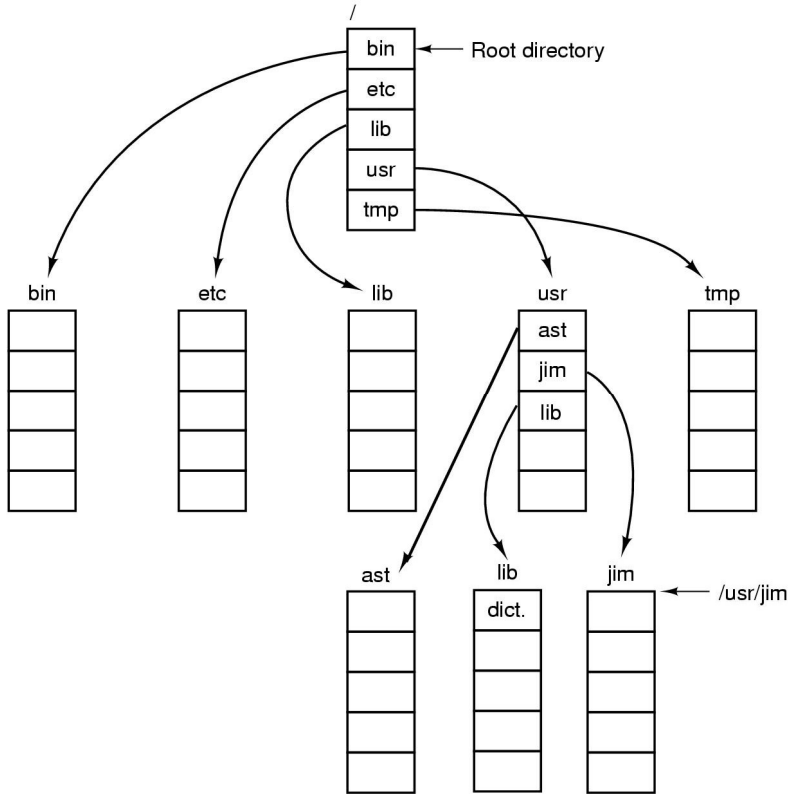
A single-level directory system containing four files

Hierarchical Directory Systems



A hierarchical directory system

Path Names



A UNIX directory tree

Directory Operations

System calls for managing directories:

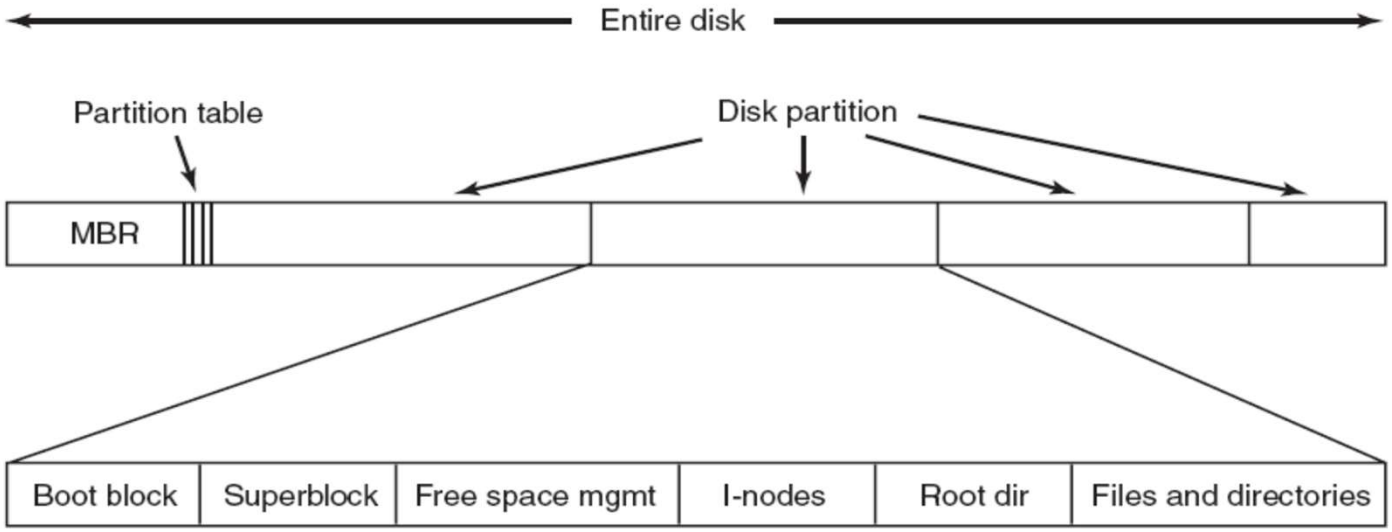
- Create
- Delete
- Opendir
- Closedir
- Readdir
- Rename
- Link
- Uplink

File Space Organization

- Disk basic allocation unit is a sector (e.g., 512 bytes)
- File system may choose to use a larger block size (e.g., 4KB)

- File allocation methods
  - How disk blocks are allocated for files
    - Contiguous allocation
    - Linked allocation
    - Indexed allocation
  - Metrics:
    - Access speed (sequential & random)
    - Space utilization

File System Layout

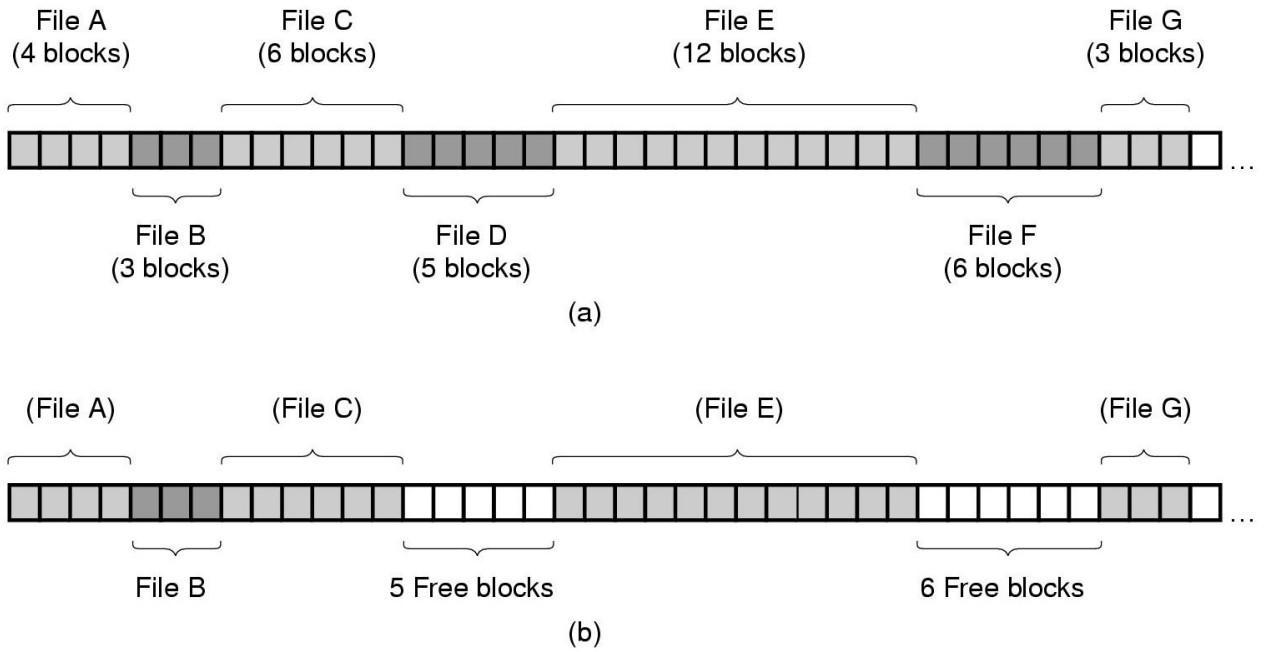


A possible file system layout

Contiguous File Allocation

- Each file occupies a set of contiguous blocks on the disk
- Advantage:
  - Simple – only starting location (block #) and length (number of blocks) are required
  - Fast sequential; also quite fast random access
- Disadvantage:
  - External fragmentation
  - Inflexible when appending to a file

Contiguous Allocation

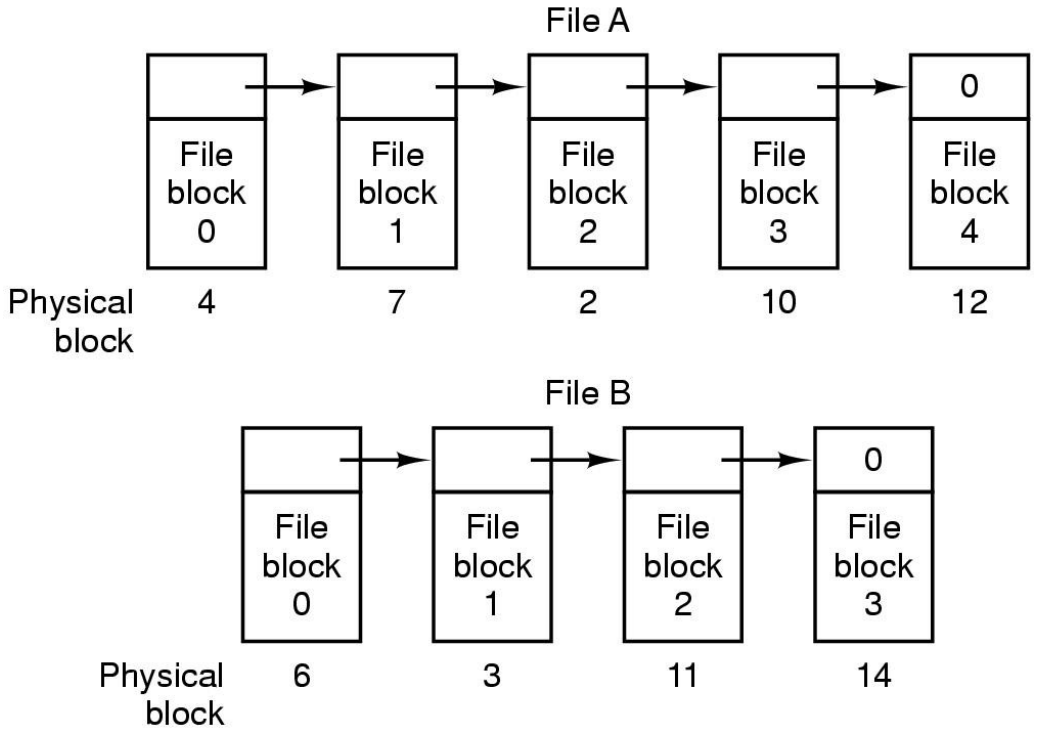


(a) Contiguous allocation of disk space for 7 files.
(b) The state of the disk after files D and F have been removed.

Linked File Allocation

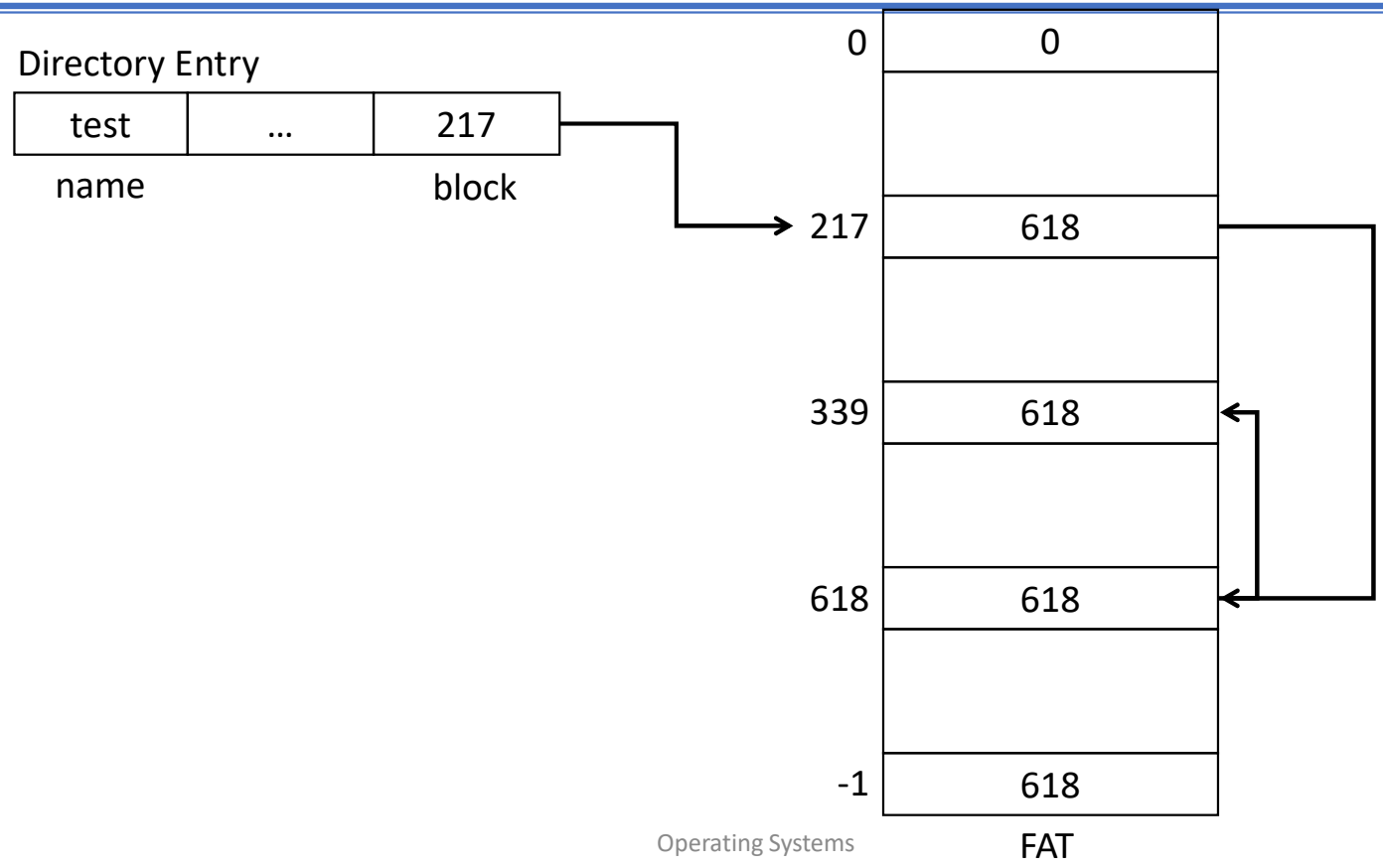
- Each file is a linked list of disk blocks
  - each block contains a next pointer
  - directory only needs to store the pointer to the first block
  - blocks may be scattered anywhere on the disk
- Advantage
  - Space efficient
  - Flexible in appending
- Disadvantage:
  - Poor access speed (sequential & random)

Linked List Allocation

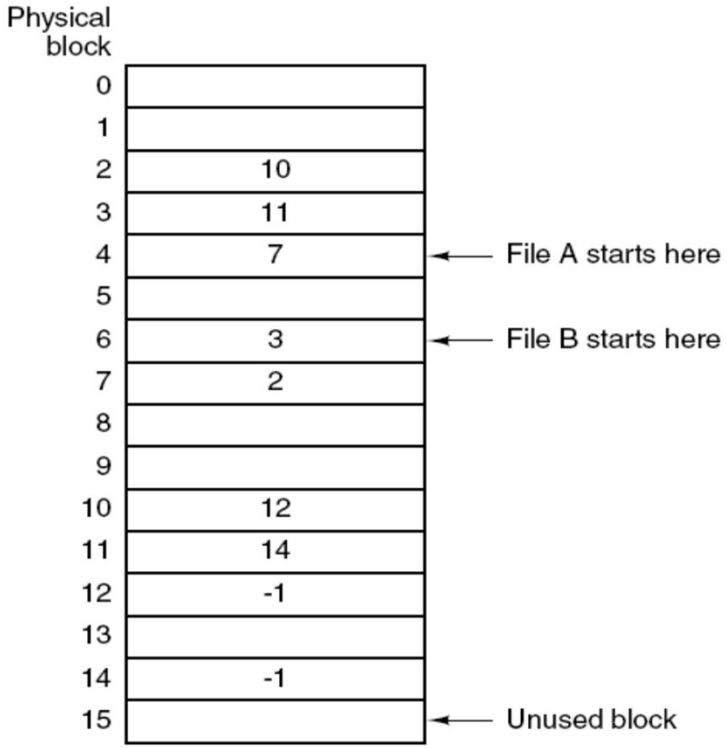


Storing a file as a linked list of disk blocks

Linked List Allocation



Linked List Allocation Using a Table in Memory

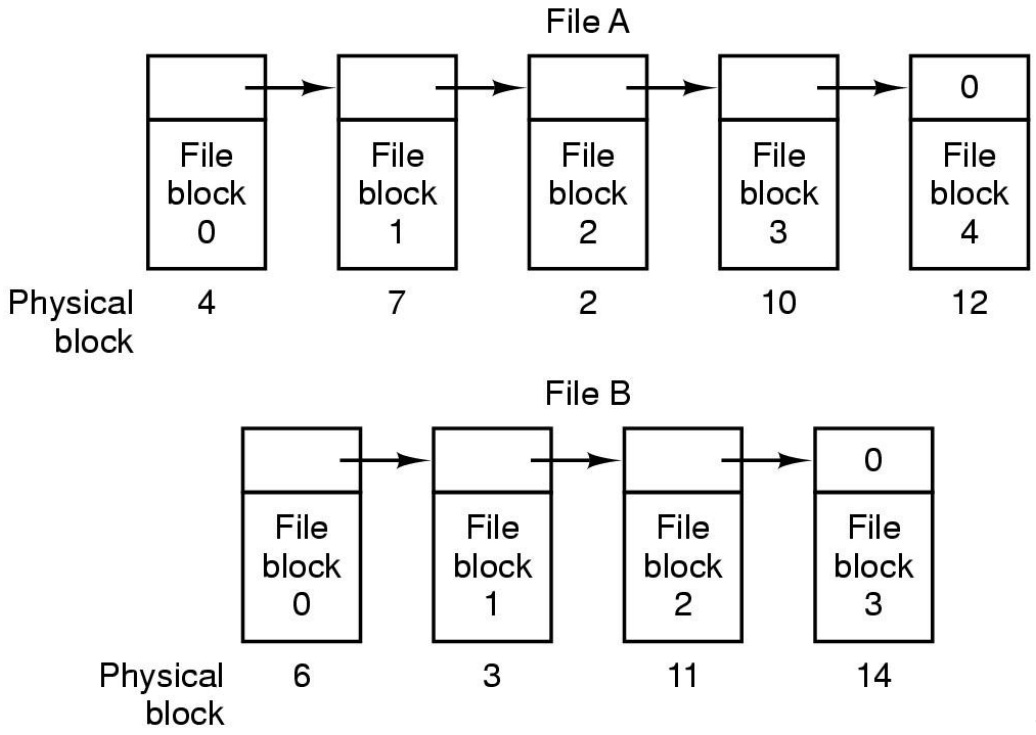


Linked list allocation using a file allocation table in main memory

Linked File Allocation

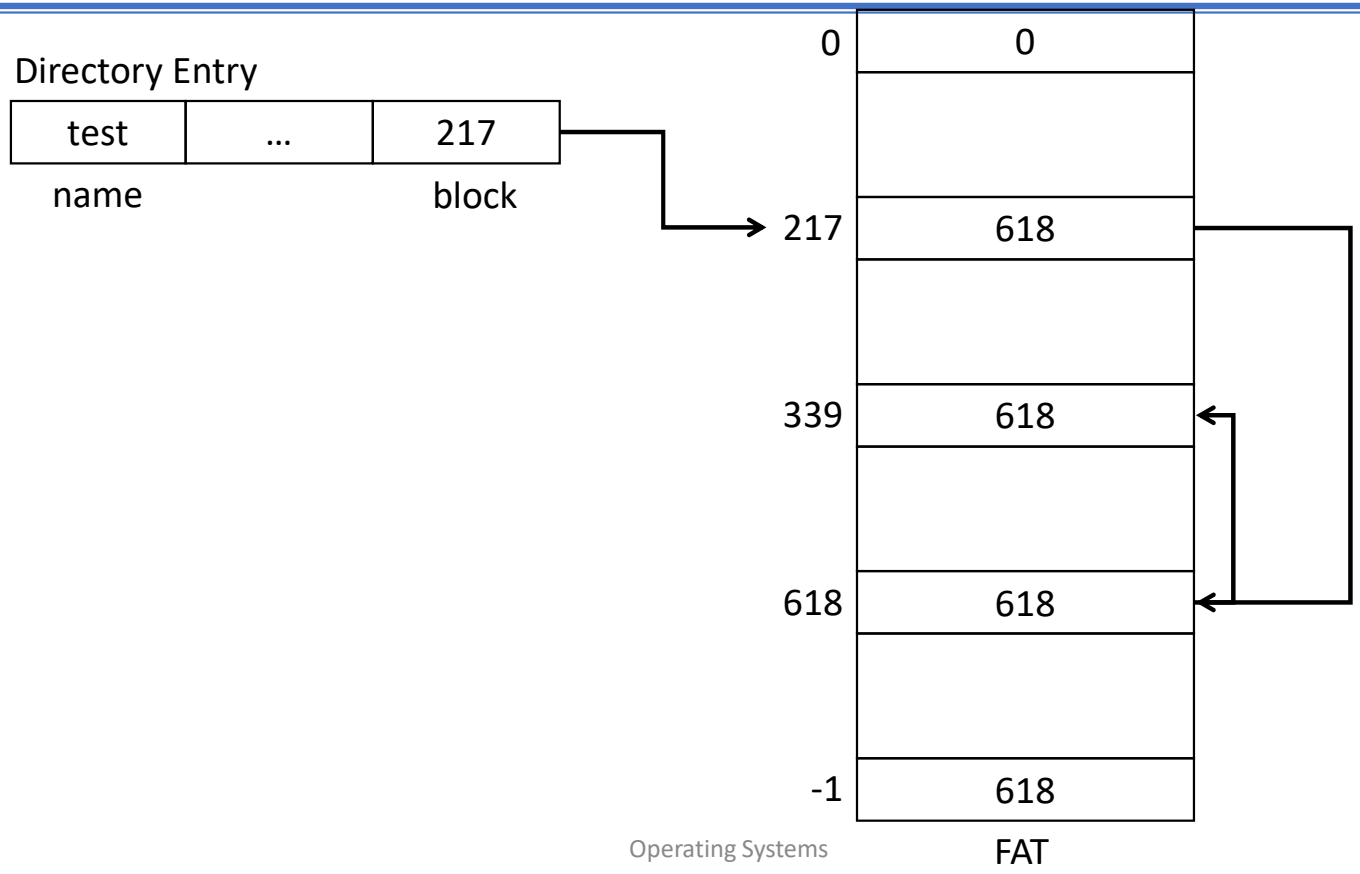
- Each file is a linked list of disk blocks
  - each block contains a next pointer
  - directory only needs to store the pointer to the first block
  - blocks may be scattered anywhere on the disk
- Advantage
  - Space efficient
  - Flexible in appending
- Disadvantage:
  - Poor access speed (sequential & random)

Linked List Allocation

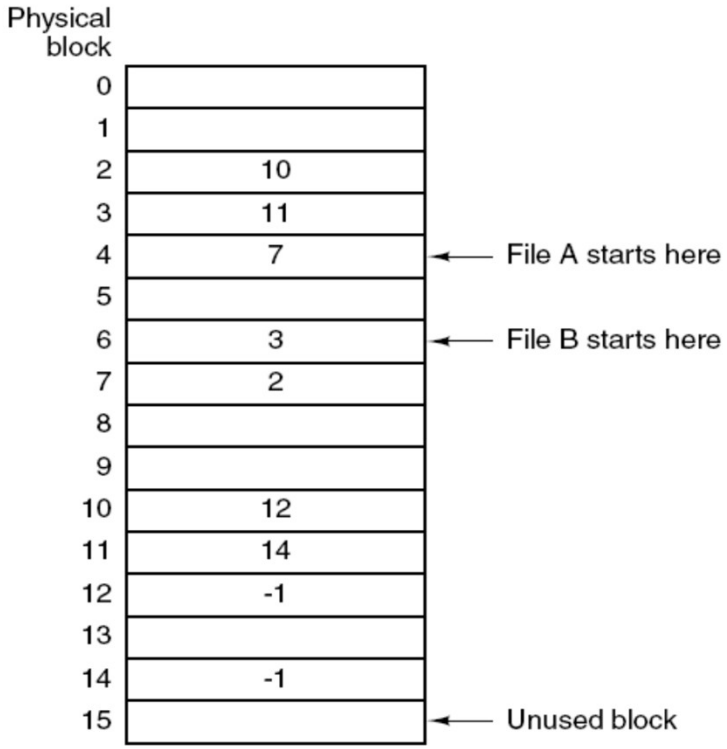


Storing a file as a linked list of disk blocks

Linked List Allocation



Linked List Allocation Using a Table in Memory



A = {4,7,2,10,12}

B = {6,3,11}

Linked list allocation using a file allocation table in main memory

File Allocation Table (FAT)

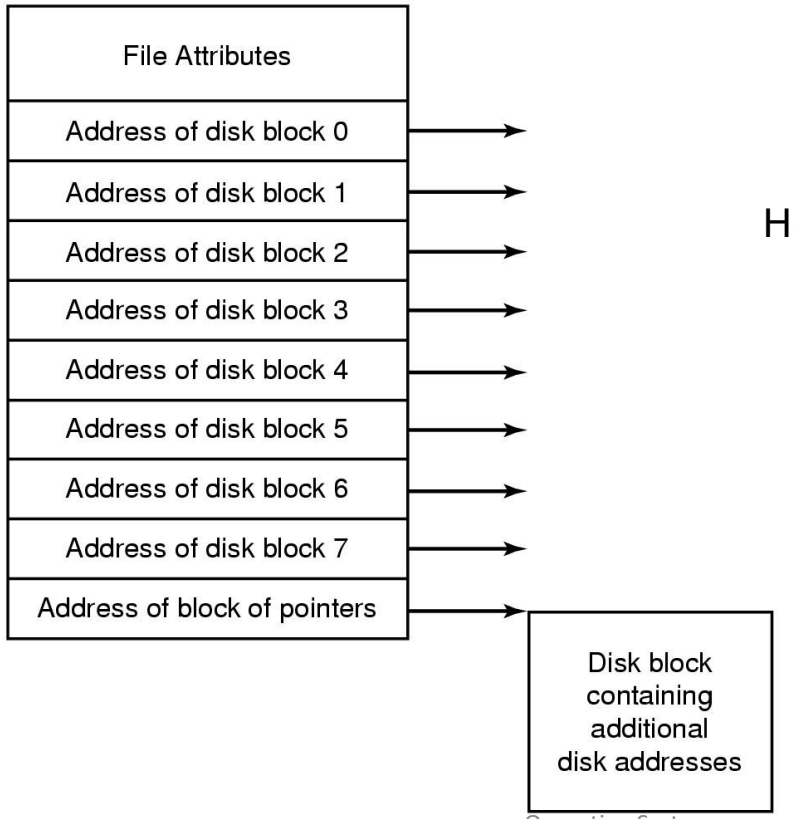
Linked List Allocation Using a Table in Memory

- FAT
- Advantage
  - Advantages of linked-list…
  - No disk access
- Disadvantage
  - Still need to traverse
  - Lot’s of memory required!
- Memory
  - 200GB disk with 1KB blocks
  - Memory required?

Indexed Allocation

- Brings all pointers together into the index block (i-node = index node)
- Just keep all attributes and addresses of each block belonging to the file in this structure
  - Keep in memory only when the file is open…
- Advantages:
  - Space efficiency
    - no external fragmentation
    - overhead of index blocks
  - Access speed
    - random access
    - sequential access

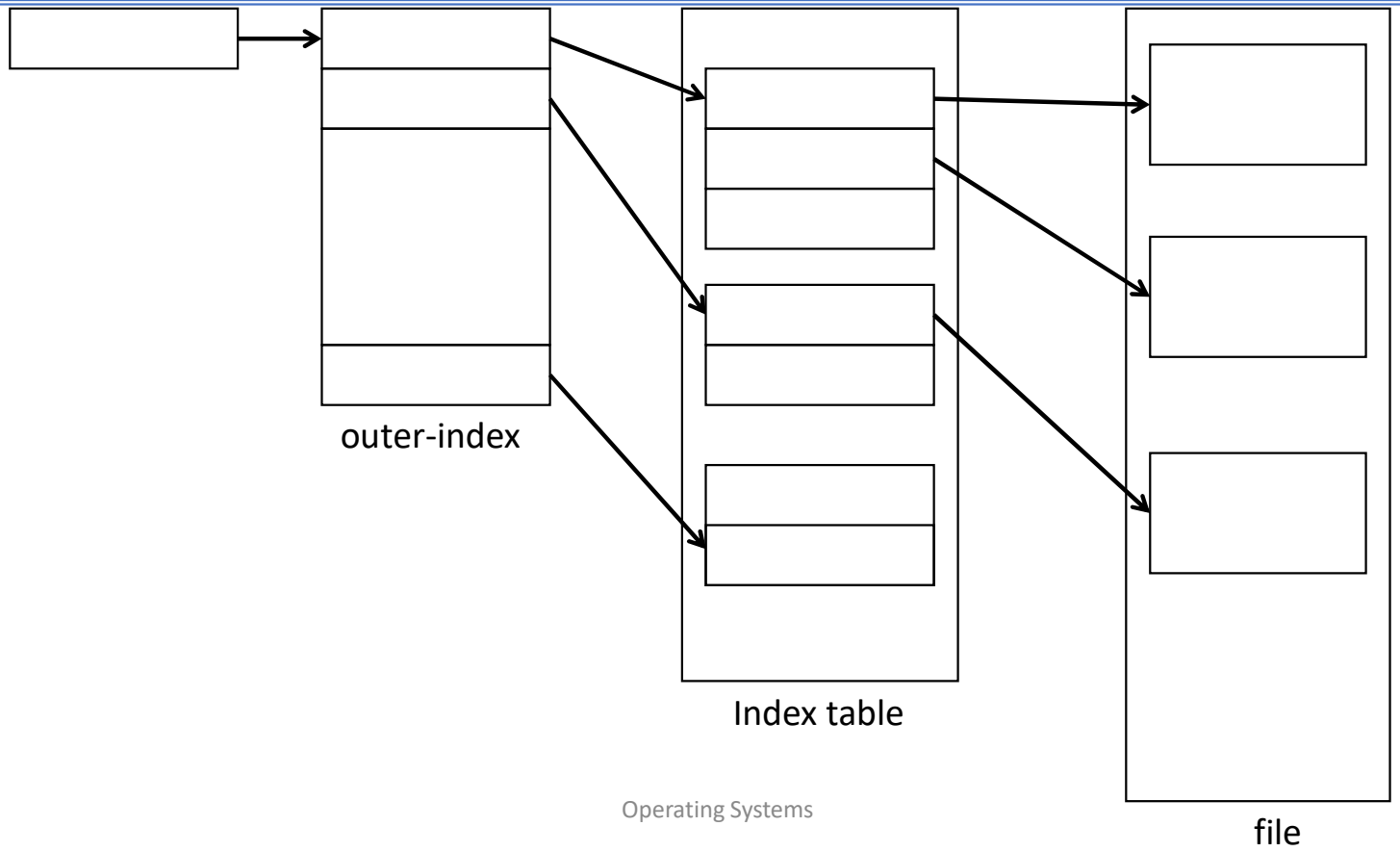
I-nodes



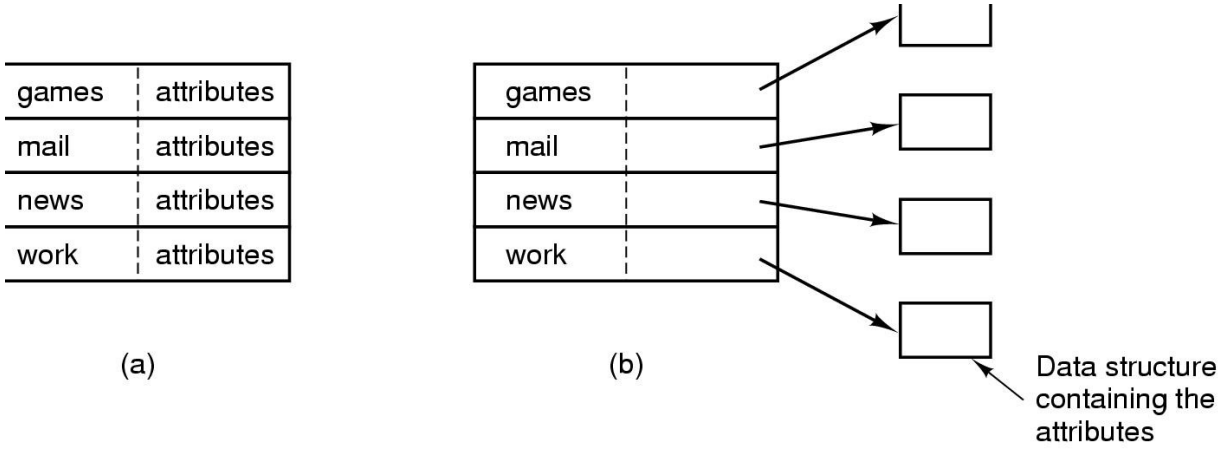
How much memory is needed?

An example i-node

Multi-level Indexed File Allocation

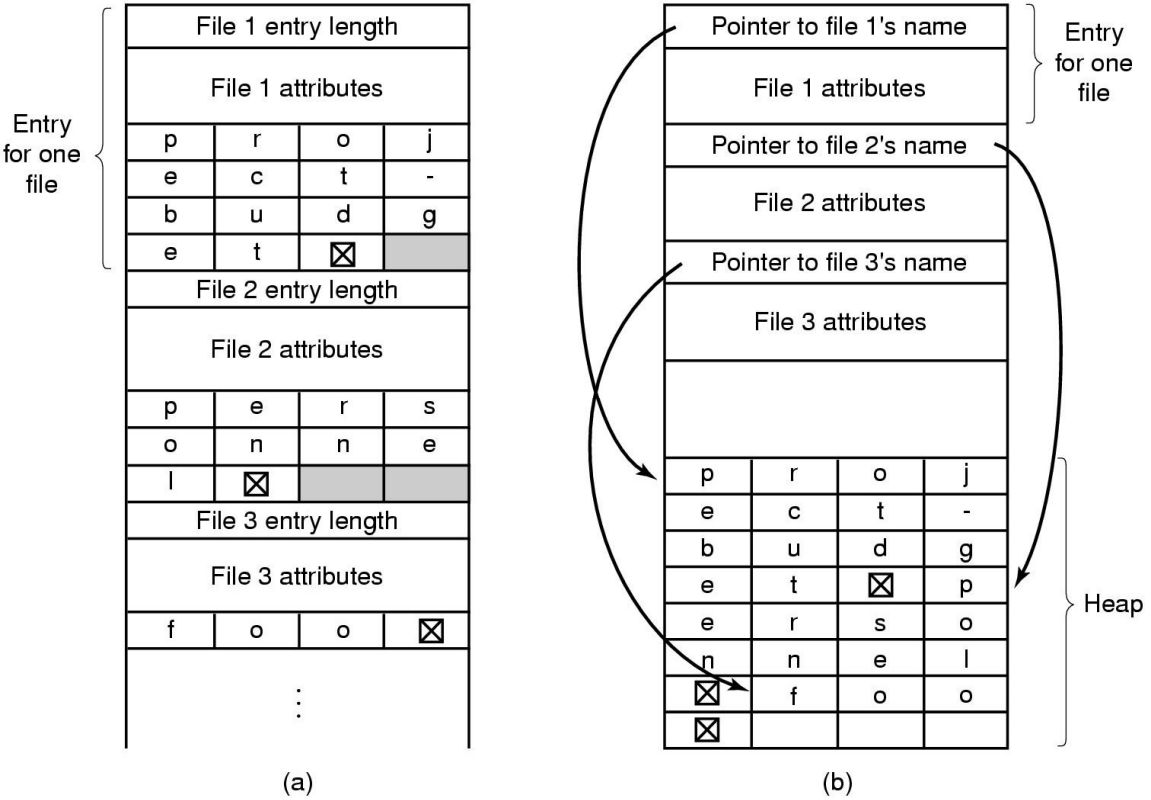


Implementing Directories



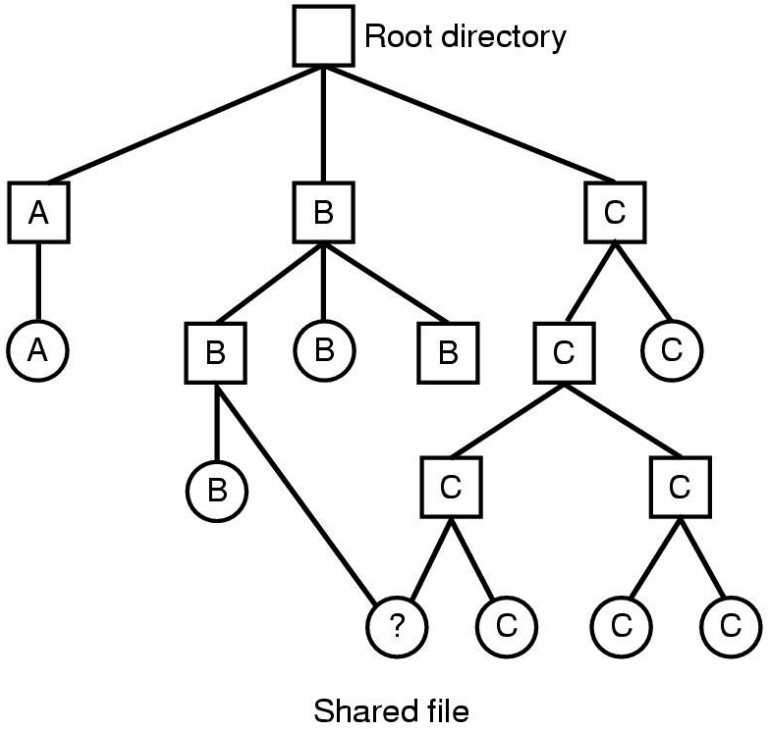
(a) A simple directory containing fixed-size entries with the disk addresses and attributes in the directory entry.
(b) A directory in which each entry just refers to an i-node.

Implementing Directories



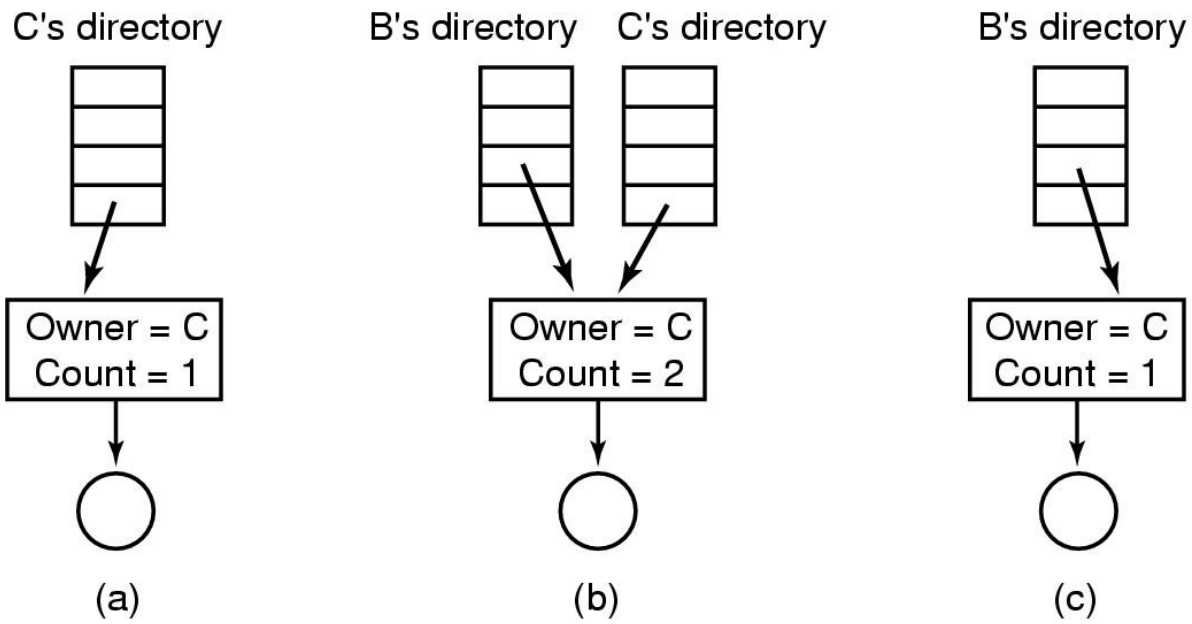
Two ways of handling long file names in a directory. (a) In-line. (b) In a heap.

Shared Files



File system containing a shared file

Shared Files



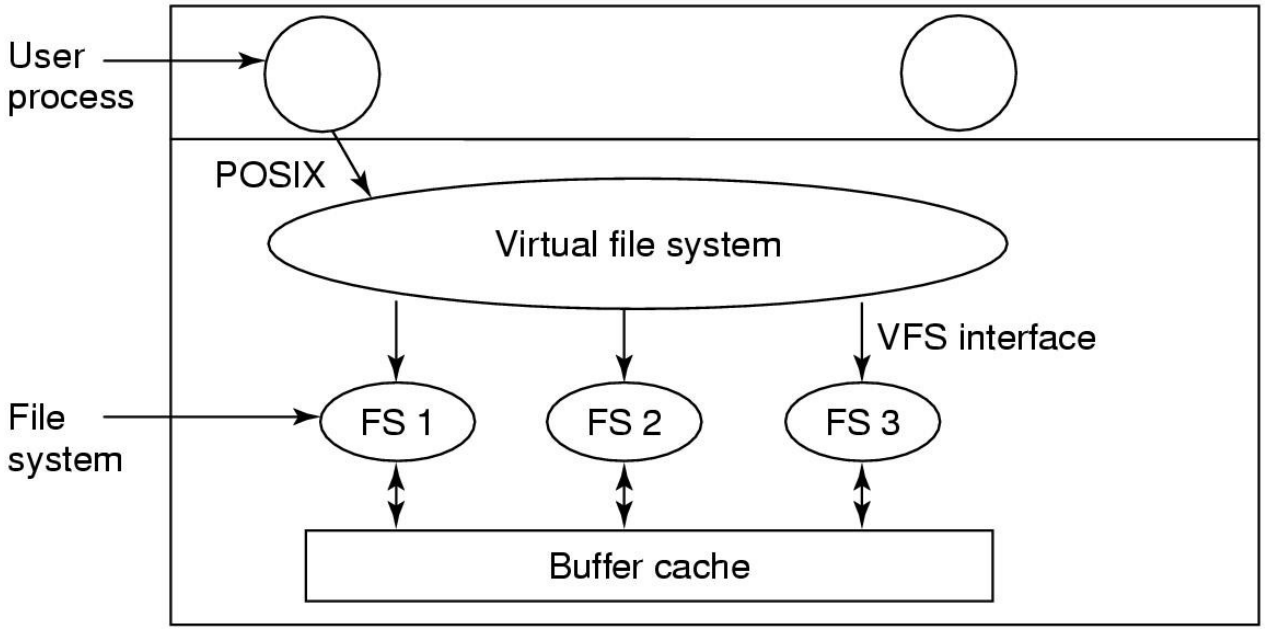
(a) Situation prior to linking.
(b) After the link is created.
(c) After the original owner removes the file.

Journaling File Systems

Operations required to remove a file in UNIX:

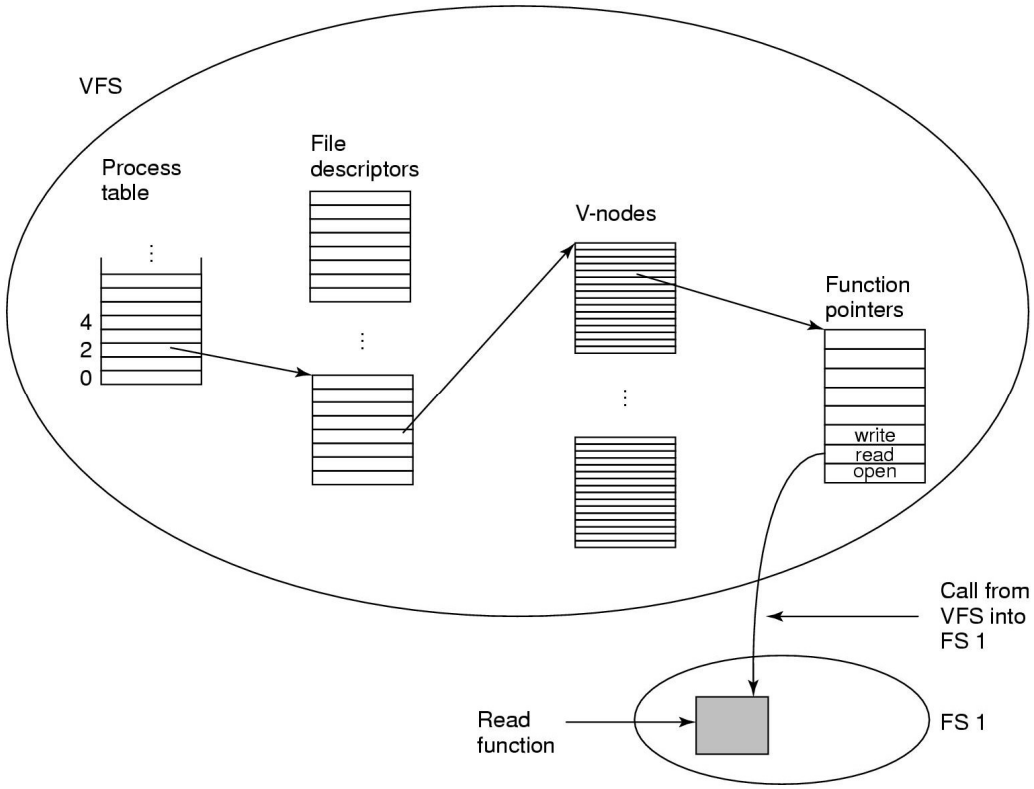
- Remove the file from its directory.
- Release the i-node to the pool of free i-nodes.
- Return all the disk blocks to the pool of free disk blocks.

Virtual File Systems



Position of the virtual file system

Virtual File Systems



A simplified view of the data structures and code used by the VFS and concrete file system to do a read

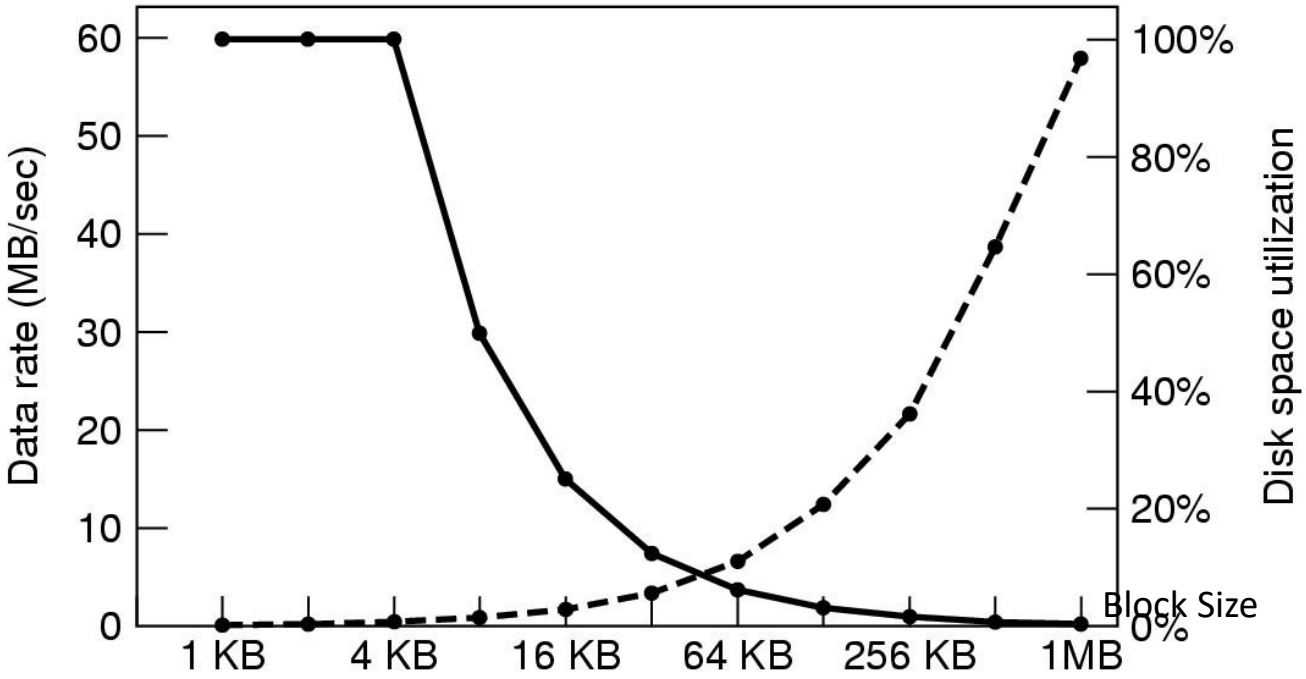
Disk Space Management Block Size

Length	VU 1984	VU 2005	Web
1	1.79	1.38	6.67
2	1.88	1.53	7.67
4	2.01	1.65	8.33
8	2.31	1.80	11.30
16	3.32	2.15	11.46
32	5.13	3.15	12.33
64	8.71	4.98	26.10
128	14.73	8.03	28.49
256	23.09	13.29	32.10
512	34.44	20.62	39.94
1 KB	48.05	30.91	47.82
2 KB	60.87	46.09	59.44
4 KB	75.31	59.13	70.64
8 KB	84.97	69.96	79.69

Length	VU 1984	VU 2005	Web
16 KB	92.53	78.92	86.79
32 KB	97.21	85.87	91.65
64 KB	99.18	90.84	94.80
128 KB	99.84	93.73	96.93
256 KB	99.96	96.12	98.48
512 KB	100.00	97.73	98.99
1 MB	100.00	98.87	99.62
2 MB	100.00	99.44	99.80
4 MB	100.00	99.71	99.87
8 MB	100.00	99.86	99.94
16 MB	100.00	99.94	99.97
32 MB	100.00	99.97	99.99
64 MB	100.00	99.99	99.99
128 MB	100.00	99.99	100.00

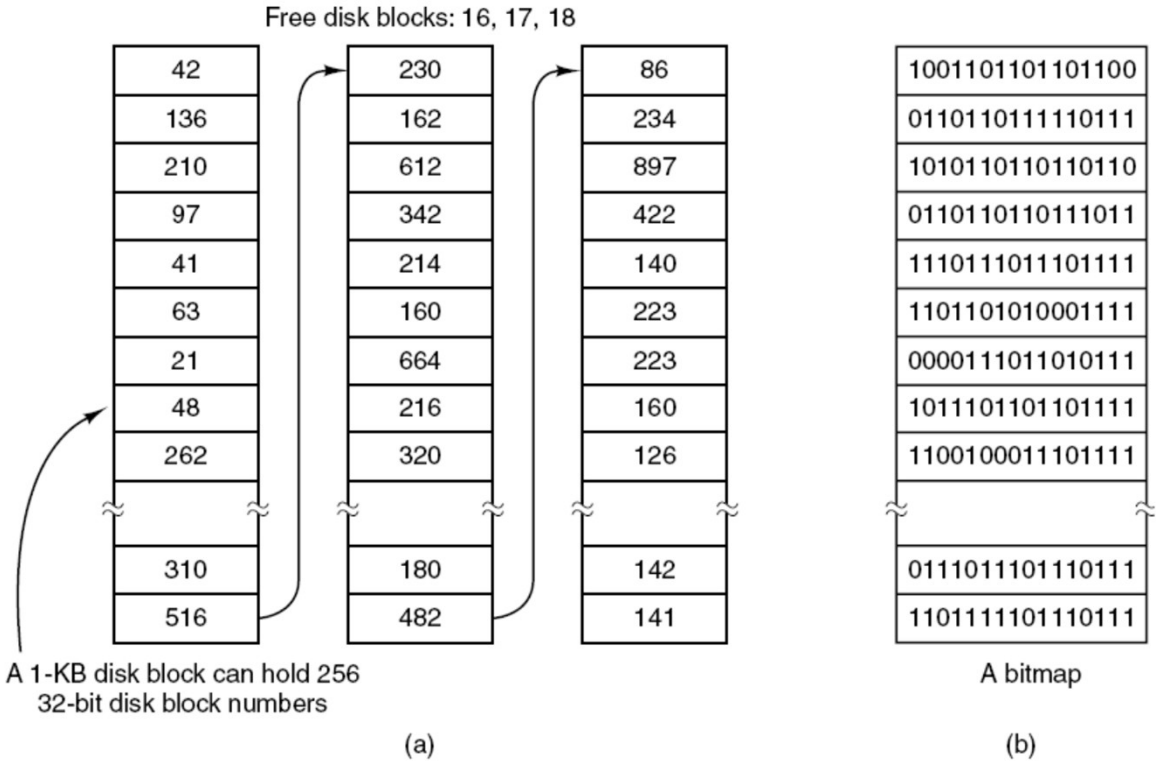
Percentage of files smaller than a given size (in bytes)

Disk Space Management Block Size



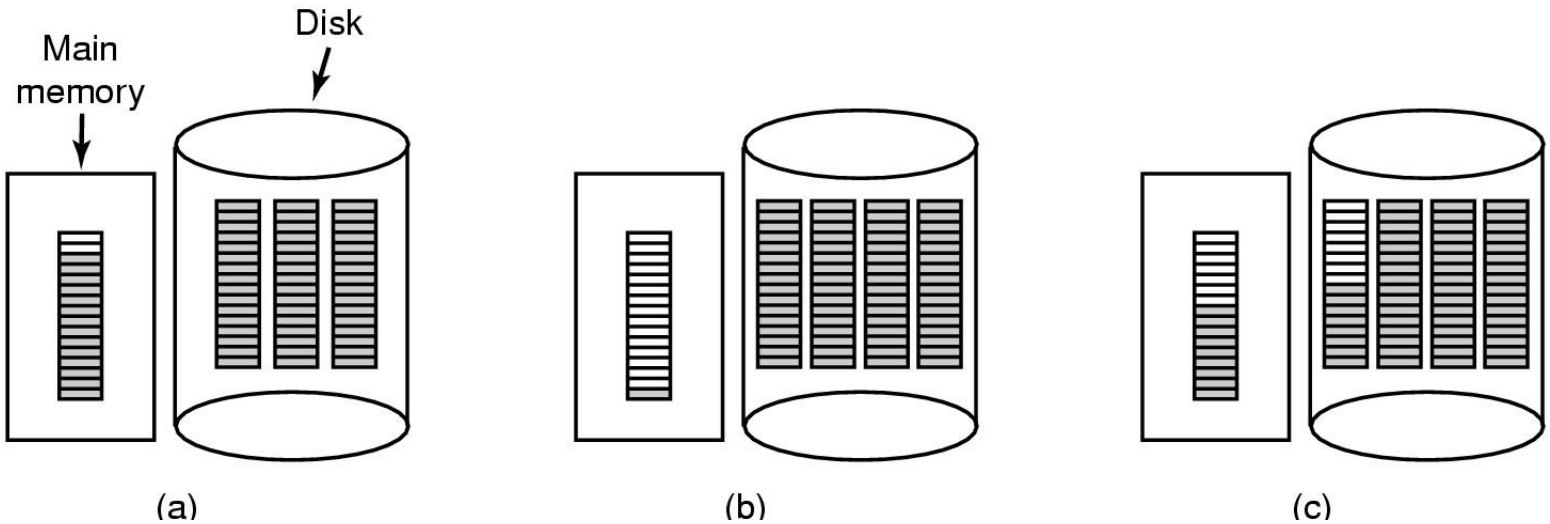
The solid curve (left-hand scale) gives the data rate of a disk. The dashed curve (right-hand scale) gives the disk space efficiency. All files are 4 KB.

Keeping Track of Free Blocks



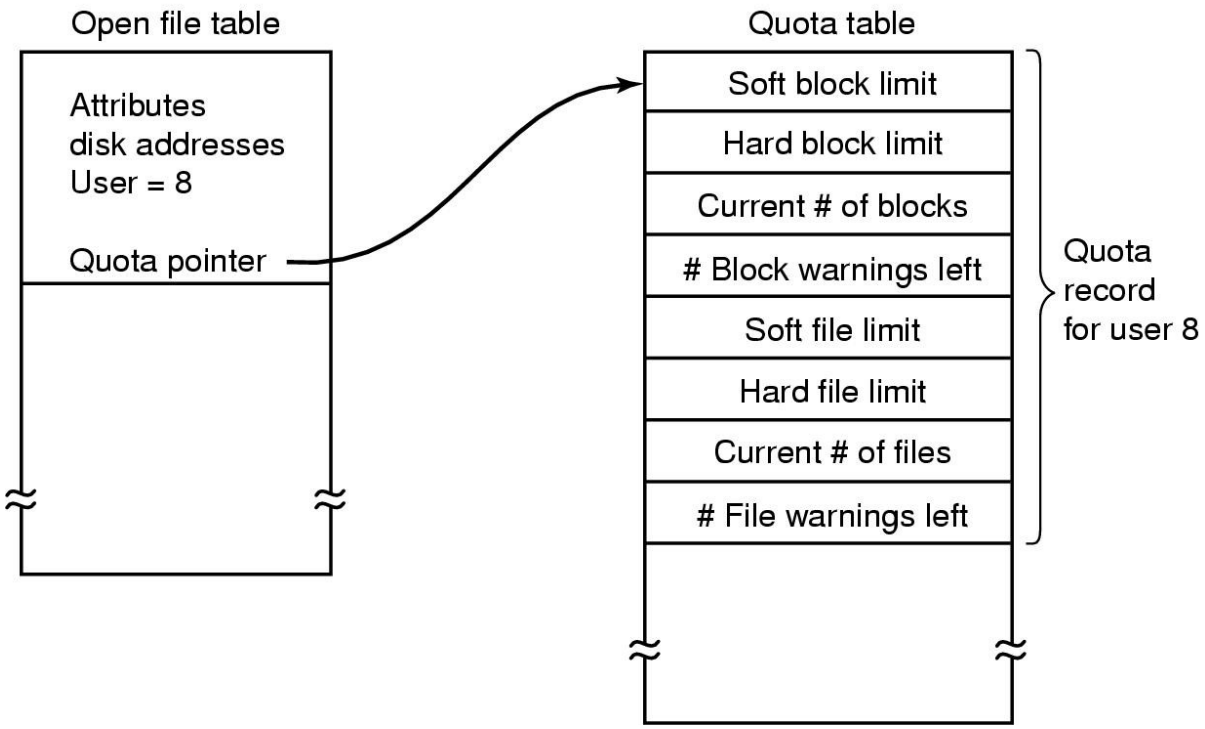
(a) Storing the free list on a linked list.
(b) A bitmap.

Keeping Track of Free Blocks



(a) An almost-full block of pointers to free disk blocks in memory and three blocks of pointers on disk. (b) Result of freeing a three-block file. (c) An alternative strategy for handling the three free blocks. The shaded entries represent pointers to free disk blocks.

Disk Quotas



ıotas are kept track of on a per-user basis in a quota table

File System Backups

Backups to tape are generally made to handle one of two potential problems:

- Recover from disaster
- Recover from stupidity

Two simple issues to consider

- Backup?
- Recovery?

File System Backups

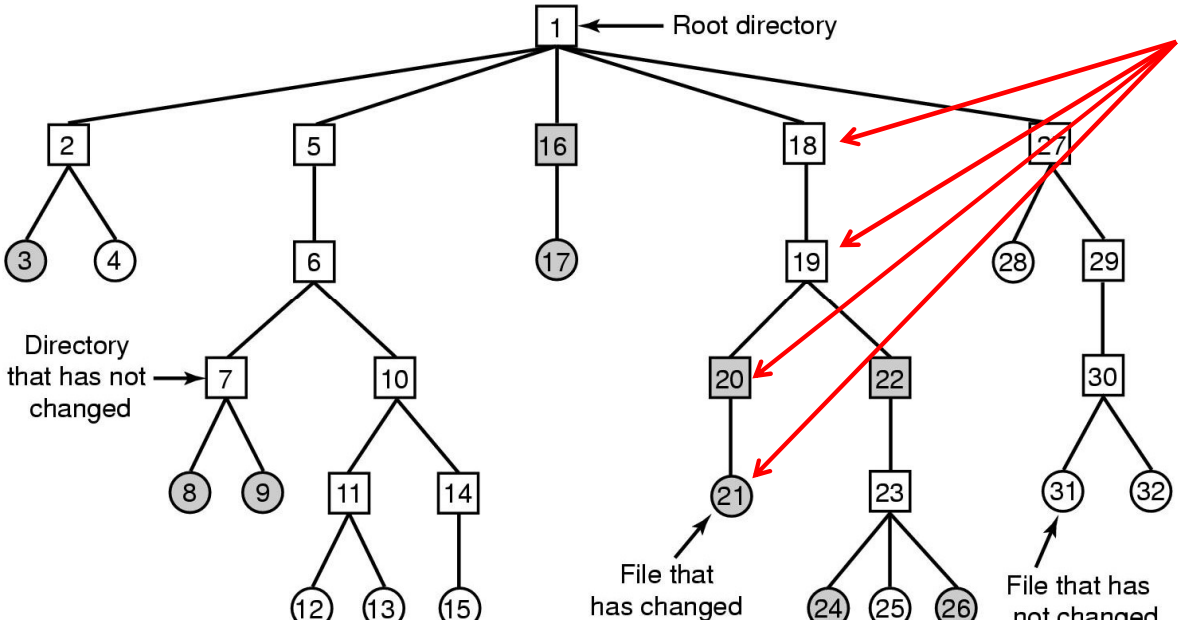
Questions:

- What to backup – entire system or only part of it?
- When to backup – unchanged, incremental dumps?
- Compression?
- Online vs. off-line backups?
- Non-technical issues: security, safety,

Dumping a Disk to a Tape

- Physical dump
  - Start from block 0,…
  - Reliable – one to one
  - Fast
  - Wasteful – free blocks, bad blocks,
- Logical dump
  - Start with one or more directories and dump files recursively …

File System Backups

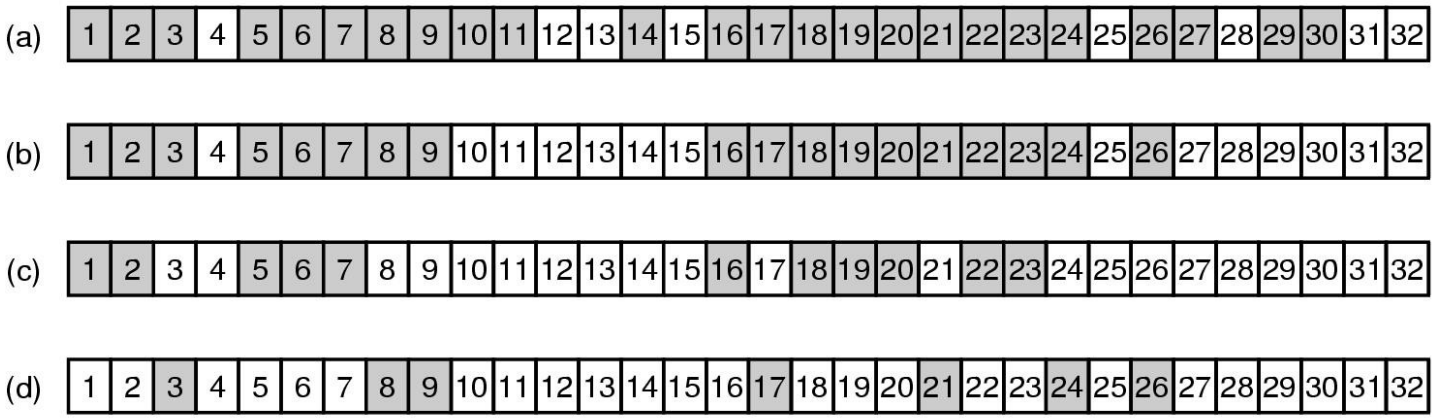


A file system to be dumped. Squares are directories, circles are files. Shaded items have been modified since last dump. Each directory and file is labeled by its i-node number.

Algorithm

- Using bitmap for each i-node (a few bits to keep track of modifications etc.)
- Phase 1: Mark all modified files and every single directory (modified or not)
- Phase 2: Unmark any directory that has no modified file in or under
- Phase 3: Scan i-nodes in numerical order and dump all directories marked for dumping
- Phase 4: Dump all the files marked

File System Backups



Bitmaps used by the logical dumping algorithm

Recovery Algorithm

- Create an empty file system on disk
- Restore the most recent full dump from the tape
  - Directories first
  - Files next
- Restore each incremental dump similarly

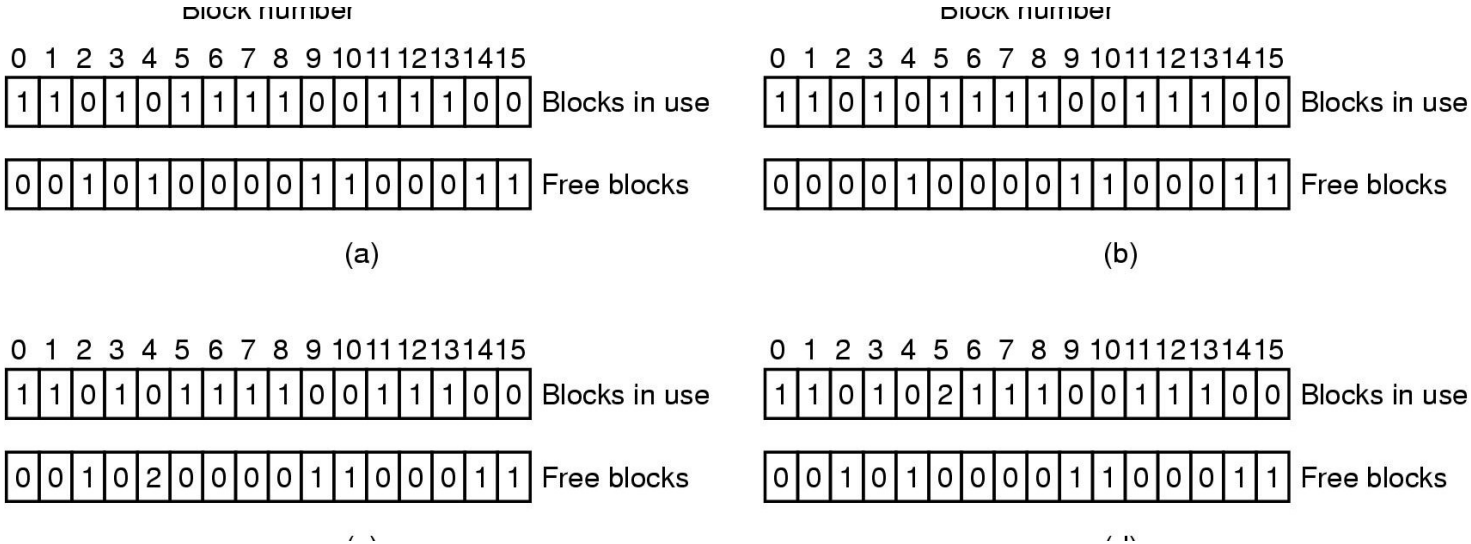
Issues

- Free blocks: Reconstruct after recovery is complete
- Links: File linked in two directories should be recovered once

File System Consistency

- “fsck” in Unix (vs. “scandisk” in Windows)
- Block consistency:
  - Keep a counter for each block in two tables
  - First table: how many times the block is contained in a file
  - Second table: how many times the block is present in the free list

File System Consistency



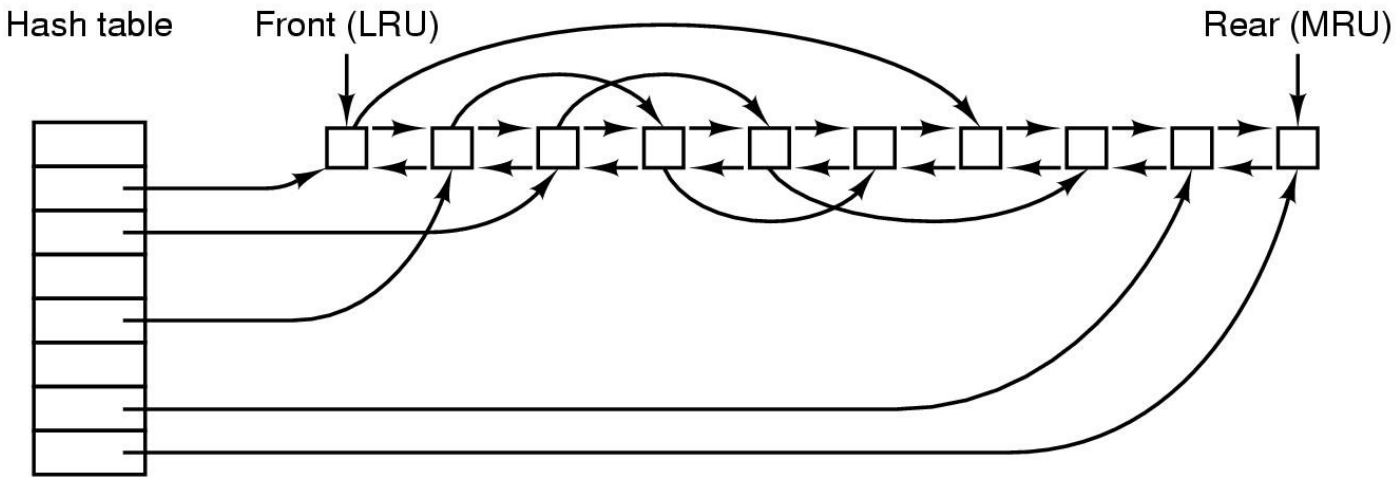
File system states. (a) Consistent. (b) Missing block. (c) Duplicate block in free list. (d) Duplicate data block.

File System Performance

Access to disk is much slower than access to memory!

- Caching
- Block read ahead
- Reducing disk arm motion

Caching

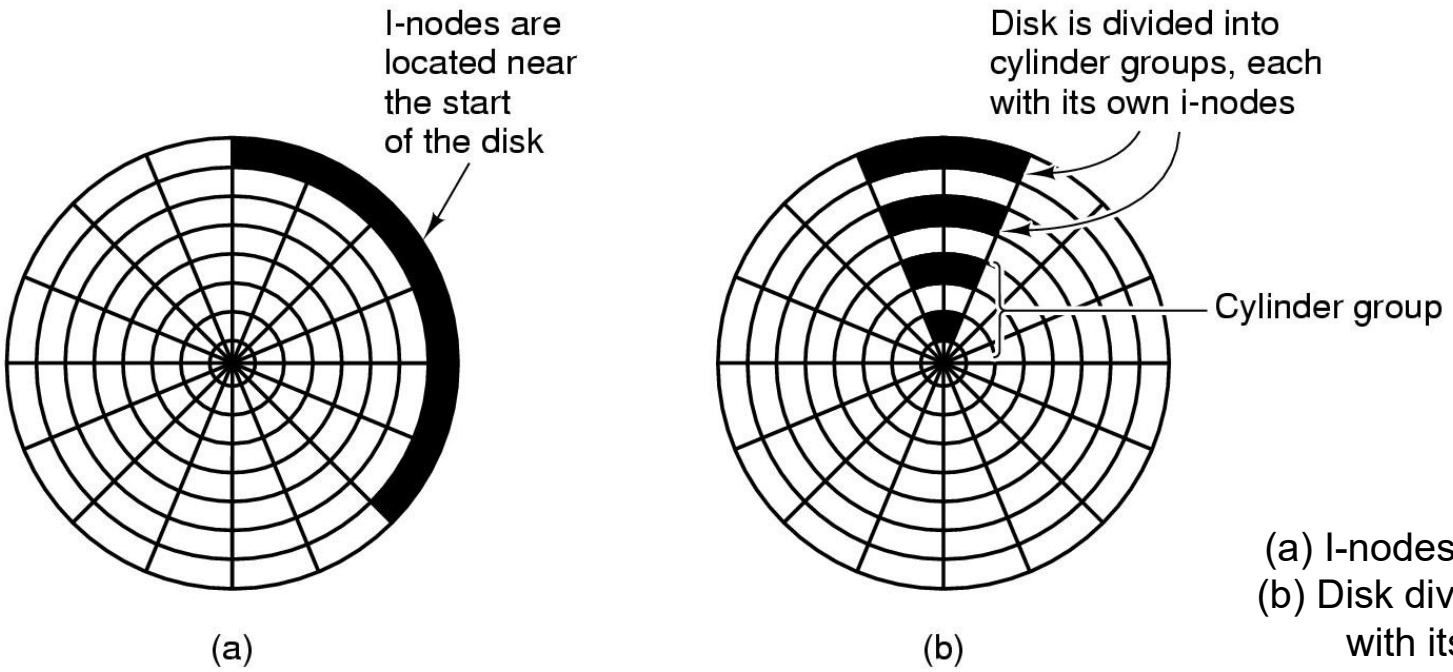


The buffer cache data structures

Caching

- Some blocks, such as i-node blocks, are rarely referenced two times within a short interval
- Consider a modified LRU scheme, taking two factors into account:
  - Is the block likely to be needed again soon?
  - Is the block essential to the consistency of the file system?

Reducing Disk Arm Motion

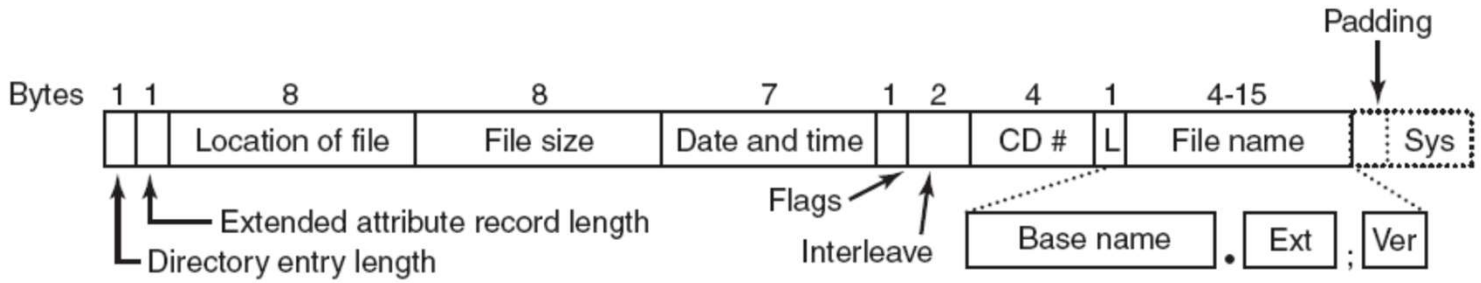


(a) I-nodes placed at the start of the disk.
(b) Disk divided into cylinder groups, each with its own blocks and i-nodes.

The ISO 9660 File System

- CD-ROM
- 16 Blocks
  - OS or …
- Primary volume descriptor
  - General information about the CD-ROM
- …

The ISO 9660 File System



The ISO 9660 directory entry

Rock Ridge Extensions

Rock Ridge extension fields:

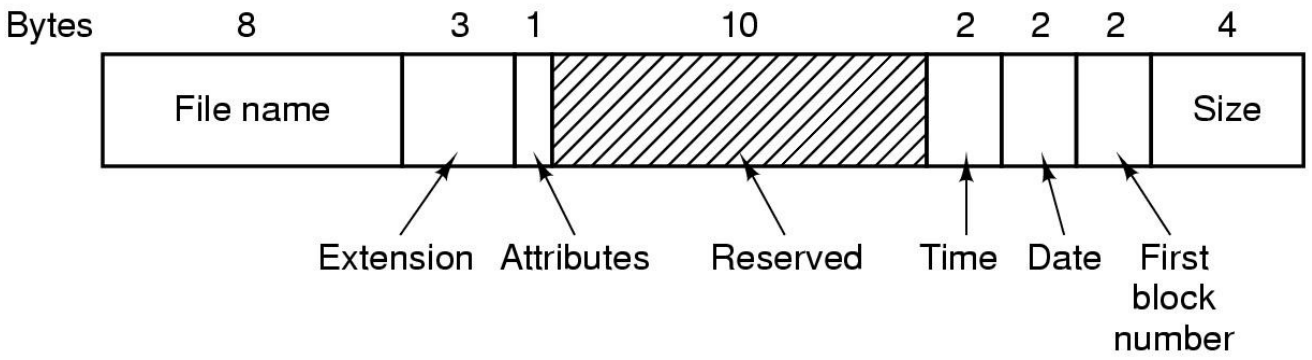
- PX - POSIX attributes.
- PN - Major and minor device numbers.
- SL - Symbolic link.
- NM - Alternative name.
- CL - Child location.
- PL - Parent location.
- RE - Relocation.
- TF - Time stamps.

Joliet Extensions

Joliet extension fields:

- Long file names.
- Unicode character set.
- Directory nesting deeper than eight levels.
- Directory names with extensions

The MS-DOS File System



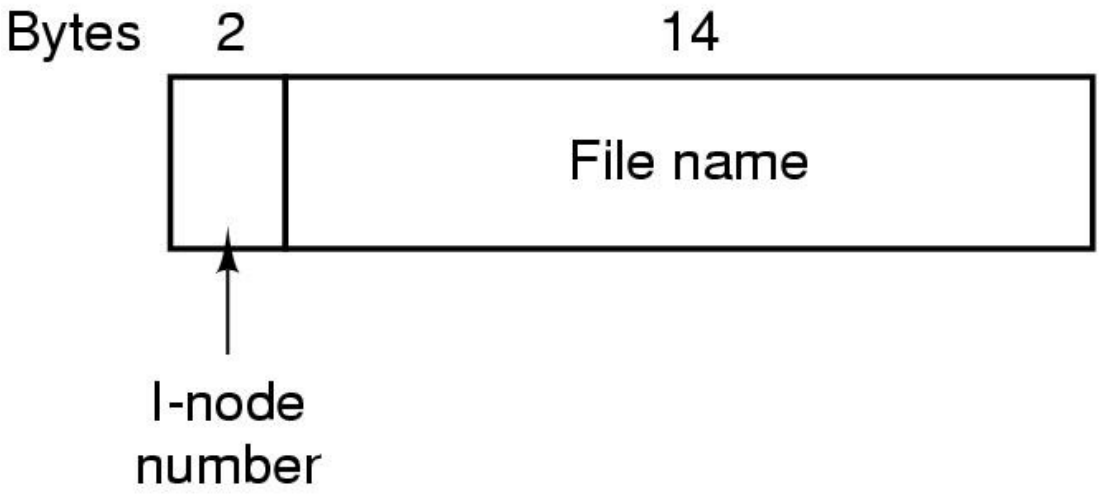
Maximum partition size for different block sizes. The empty boxes represent forbidden combinations

The MS-DOS File System

Block size	FAT-12	FAT-16	FAT-32
0.5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

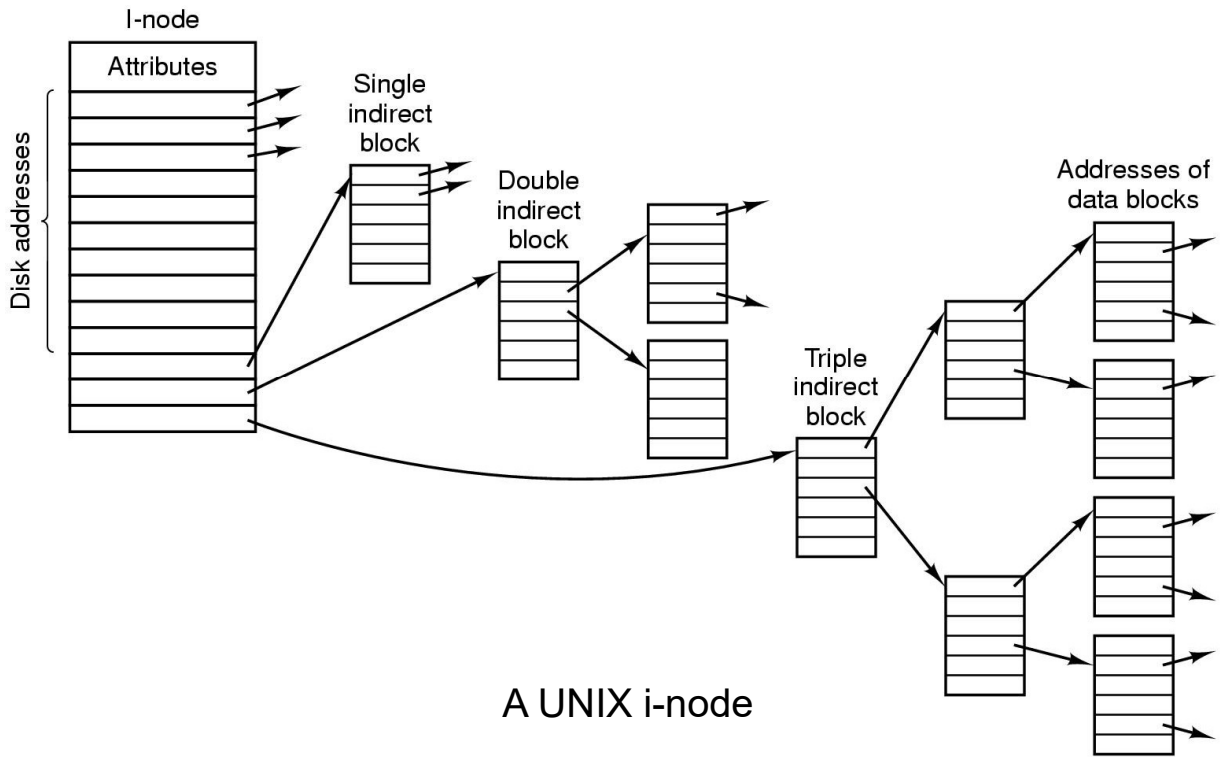
Maximum partition size vs block size…

The UNIX V7 File System



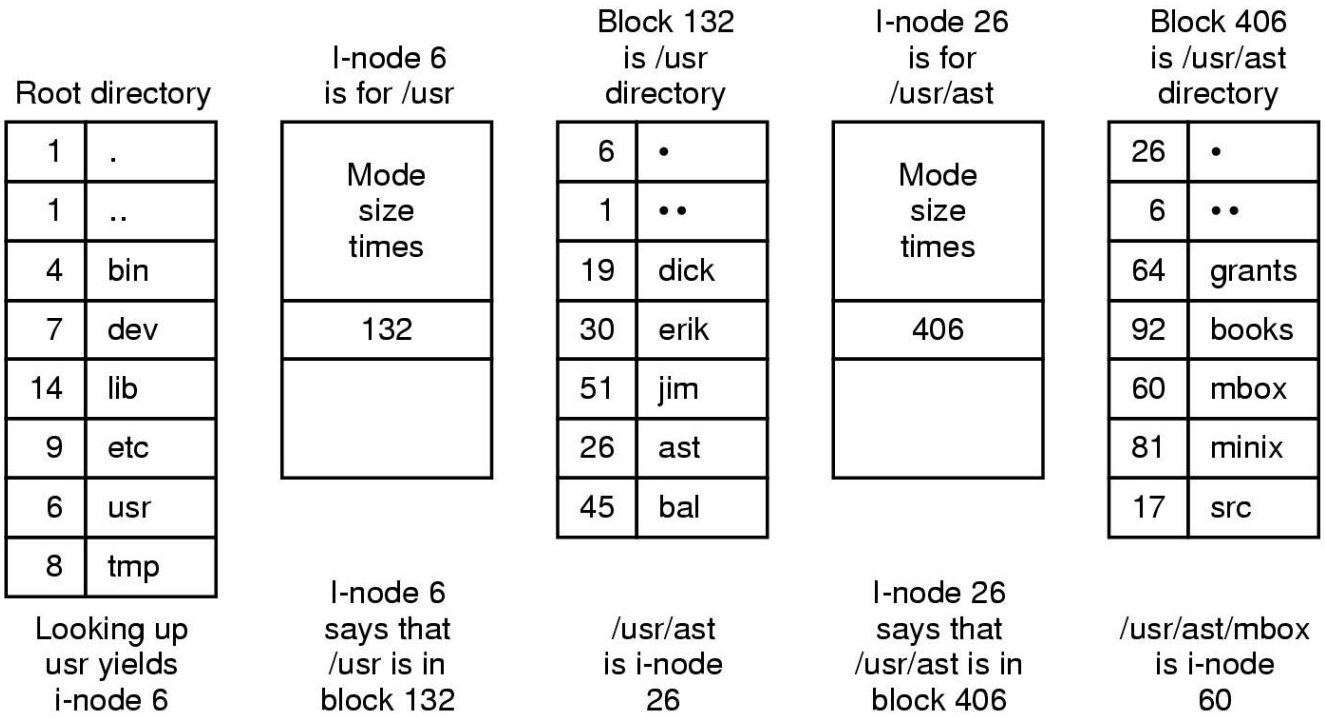
A UNIX V7 directory entry

The UNIX V7 File System



A UNIX i-node

The UNIX V7 File System



The steps in looking up */usr/ast/mbox*

The Unix Filesystem

- A high-level look at the Unix file system
- Similar to Windows and others at a high level
- Note: all modern operating systems support multiple file system types; differences hidden by abstraction layer

From the Process

- The process has two crucial directory attributes, the current root and the current working directory
- Paths that start with a / (known as absolute paths) start from the current root; those that do not start with a / (relative paths) start from the current working directory
  - Roots other than the real root are a security mechanism; we'll discuss that later

Directories

- On some Unix systems, directories can be read like any other file with read(); on Linux, you must (and on other Unix systems you should) use readdir()
- A directory entry consists of a variable-length name and an i-node number
- What's an i-node?
- By convention, on most Unix systems the first two entries in a directory are . and .. — the current and parent directories
- Don't count on them being there; they're not guaranteed by the spec!

Finding a File

- Find the next component in the pathname
- Read the current directory looking for it
- If there's another component to the path name and this element is a directory, repeat, starting from this element
- When we're done, the result is an i-node number

. and ..

- Note how . and .. work
- . has the i-node number of the current directory — you start again from this node for the next component
- It's just another directory; to (this part of) the kernel, there's nothing special about it
- The same is true for .. — it "happens" to point up a level in the directory tree.
- Following a search path does not rely on the directory structure being a tree! That's primarily needed for orderly tree walks.
- (Mental exercise: symbolic links do introduce the possiblity of loops. How can this be dealt with?)

I-Nodes

What's an i-node?

- An i-node holds most of the metadata for a Unix file — on classic Unix, it holds all but the i-node number itself
- The i-list is a disk-resident array i-nodes
- The i-node number is just an index into this array
- Looked at another way, a directory entry maps a name to an array entry
- Files are actually described by i-nodes, not by names

What's in an I-Node?

- All of the fields from the stat structure
- A few other pieces of user-settable metadata (flags)
- Disk block information — where on disk the file resides?
- How many blocks is enough?
- Put another way, how big can a file be?

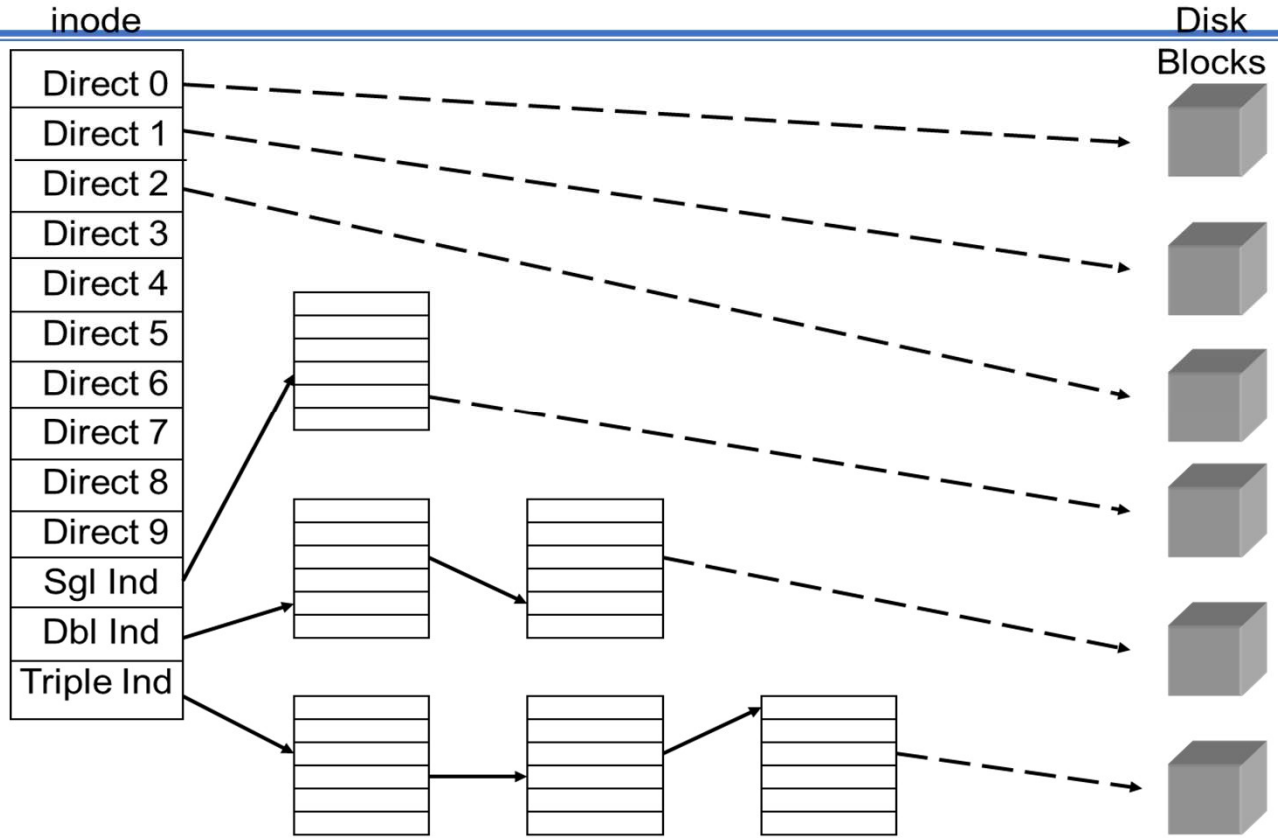
Disk Blocks in the I-Node

- If we have a small array, we limit the size of a file too much
- If we have a large array, we waste space in the i-list, because most files aren't huge
- We have a modest-size array of block addresses, followed by the address of an indirect block
- The indirect block is an array of disk addresses
- Suppose the i-node points to ten 4K blocks, followed by an indirect block. The maximum size of a file is then $4096 \times 10 + (4096/4) \times 4096$
- That's 4,235,264 bytes – not nearly big enough

Multiple Layers of Indirection

- “Any problem in software can be solved by adding another layer of indirection” —David Wheeler
- The second indirect block is a double indirect block
- That gives us 4096 × 10 + (4096/4) × 4096 + ((4096/4)×4096/4)×4096 bytes — about 4G
- Is that enough? Some systems today have triple indirect blocks. . .
- Metanote: PDP-11 Unix had triple indirect blocks; when file system blocks became 4K (more or less), there was no need for them. But disks and files grew bigger. . .

Direction and Indirection



Opening a File

- When a file is opened, the i-node is read into memory (if necessary) and its reference count is incremented
- A file table entry is created for it
- The index in the file table is passed back to the application as the file descriptor
- Virtually all operating systems have this notion — a file handle — that is a short way of referring to an open file.
- More complex for special files — wait a few days

Creating a File

- See if there's a directory entry.
- If there is, it's like opening a file (but you may have to truncate it)
- If there's no entry, create one.
- That involves writing to the directory, which is a lot like writing to a regular file
- It also involves finding and allocating a free i-node
- Directory entries are free if the i-node number is 0; i-nodes are free if the link count is 0

Reading a File

- Convert the current byte offset to a block number
- Read that block from the disk
- Pass the proper bytes back to the user
- Update the current byte offset
- Optional: if access to the file appears to be sequential, start — but don’t wait for — the read of the next block
- Get it in the buffer cache ahead of use, to improve performance

Writing a File

- Are we writing to the middle of a block?
- If so, read that block in
- If not, allocate a new block from the freelist and add it to the i-node
- Copy the data from the user to a buffer
- Mark that buffer dirty, so that it will (eventually) be written out
- Update the file offset pointer

Seek

- Simply change the current byte offset
- Does not actually move the disk arm
- Doing that is probably pointless on a multitasking system

Closing a File

- Decrement the i-node's reference count
- Delete the file table entry
- If the i-node's link count is 0, the file has been deleted; see below
- Everything else is automatic

Linking to a File

- Create a new directory entry
- But — the i-node number is the number of the existing file
- Increment the i-node's link count

Unlinking a File

- Decrement the link-count
- If the link-count is still non-zero, the file has other names; do nothing more
- If the link-count is now 0 (and the in-memory reference count is 0), the file is being deleted
- Return all of its blocks to the free list
- (Note: you can unlink an open file; it isn't actually deleted until it's closed. Query: what happens if the system crashes with a file in that state?)

Updating Metadata

- When a file is read, the access time needs to be updated
- When a file is written, the modified time needs to be updated
- If the user changes things like file permissions, make the appropriate changes
- Mark the in-memory i-node as dirty
- Also update the i-node change time
- Periodically, dirty i-nodes are rewritten to disk

Creating Directories

- Similar to creating a file
- But — the kernel first writes the . and .. entries
- Increment the link count in the parent directory — .. points to it
- (All writes to directories are done by specialized system calls; user programs cannot write directories directly via write() on any modern Unix)

Deleting Directories

- First make sure the directory is empty except for . and ..
- Then delete it the same way a normal file is deleted
- But – the link count in the parent directory is decremented
- It's possible to delete the current working directory, or even its parent!

Renaming

- Create a link with the new name
- Remove the link with the old name
- But — it must be done in the kernel, since the first step involves creating a hard link to a directory

Components of a File System

- Boot record, superblock
- i-list
- Freelist
- Data area

Boot Record and Superblock

- The boot record is at the start of the disk; it's used by the BIOS for booting (called the Master Boot Record (MBR) on PCs)
- This area also stores the disk label — information on how the disk is partitioned
- Next is the superblock — contains essential file system parameters
- Among other things: how to find the i-list; the "clean shutdown" flag, to indicate that the file system is believed to be in a consistent state

The I-List

- Once, the i-list was an array of blocks just after the superblock
- Today, it's distributed — a piece of the i-list is in each cylinder group
- A cylinder group contains a portion of the i-list, a freelist for blocks within the cylinder group, and a data area
- Newly-created files get their initial block allocations within the cylinder group; later blocks are allocated in groups in other cylinder groups
- What is the purpose of cylinder groups?
- Locality of reference — try to avoid long seeks

Data Area

- Most of the file is allocated in blocks of 4-8K bytes
- Left-over pieces of the file are stored in fragments, which are composed of 512-byte blocks
- Dual block size saves RAM and disk space for large files, but doesn't waste too much for short files

The Windows FAT File System

- Limited-size root directory
- 8.3 file names
- Metadata in the directory entry
- Directory points to FAT table entry

File Allocation Table

- The FAT keeps track of allocated blocks
- Each entry in the FAT — on disk and in RAM — is just a pointer to the next block
- Implements a linked list of blocks, without the need to read each block
- Maximum file size limited by number of bits in a disk block address — current is using 28-bit addresses

Supporting Long Names

- For Windows 98, they wanted to support longer names than 8.3 permitted
- But — must maintain substantial name compatibility with older versions of Windows and DOS
- First: use recognizable part of long name for 8.3 version
- Second: create fake, invalid directory entries that precede the real one
- DOS will ignore them, because they appear invalid
- Have a checksum in case the real, short-name version of the file is deleted while running DOS

Dumping a Disk

- Must have a way to dump and restore disks — some people make backups
- More than — we want a way to to create incremental dumps: all files changed since the last dump
- Unix has multiple levels of backup: 0 is everything; 1 is everything since the last level 1; 2 is everything since the last level 1; etc.
- Can select files by date modified or by a “dirty” bit
- Some systems have a way to exclude some files from being dump (i.e., swap files or very sensitive files)

Dump Strategies

- Could do a physical image dump
- Safe and simple, but wasteful — dumps empty blocks, can’t do incremental dump, might try to dump bad blocks, etc.
- Only restorable to disk with identical geometry
- Instead — dump a filesystem

Mapping a Filesystem

- Must map the file system to see what files must be dumped
- For incremental dumps, must be able to “dump” deletions — those are changes to the parent directory
- Must also dump all parent directories, up to the root, of any dumped files
- Dump selection is based on metadata (and metadata must be dumped)

Dumping a Filesystem

- The actual dump can’t go through the file system on Unix; want to avoid dumping file system “holes”
- The actual dump file is based on i-node numbers, not file names; the file names are in the dumped directories
- Restores can be done through the file system
- To do incremental restores, must restore each level in sequence, to build the proper directory structure

See you next time!